

BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÀI TIỂU LUẬN MÔN HỌC
TÊN HỌC PHẦN: PHÁT TRIỂN HỆ THỐNG THÔNG MINH

Giảng viên hướng dẫn: *Trần Đình Quế*
Sinh viên thực hiện: *Nguyễn Hoàng Tùng*
Mã sinh viên: *B22DCDT294*
Lớp: *E22CNPM03*
Hệ: *Đại học chính quy*

Hà Nội, 2025

PHẦN 1 – 15 CÂU HỎI LÝ THUYẾT TỔNG HỢP

1. Deep learning là gì? So sánh deep learning với machine learning truyền thống.

- Deep Learning (Học sâu) là một nhánh cụ thể và là một phương pháp mới trong lĩnh vực Machine Learning (Học máy). Machine Learning lại là một lĩnh vực con của Trí tuệ Nhân tạo (Artificial Intelligence - AI).

- Về cơ bản, Deep Learning tập trung vào việc học các lớp biểu diễn kế tiếp nhau (successive layers of representations), ngày càng trở nên có ý nghĩa hơn từ dữ liệu.

Khái niệm "Deep" (Sâu)

- Cụm từ "deep" (sâu) trong Deep Learning không ám chỉ mức độ hiểu biết sâu sắc nào, mà thay vào đó, nó đề cập đến ý tưởng về các lớp biểu diễn nối tiếp nhau (layered representations learning hoặc hierarchical representations learning).

- Các mô hình Deep Learning hiện đại thường bao gồm hàng chục, thậm chí hàng trăm lớp biểu diễn nối tiếp nhau.
- Các biểu diễn phân lớp này được học thông qua các mô hình gọi là mạng nơ-ron (neural networks), được cấu trúc thành các lớp xếp chồng lên nhau.
- Một mạng nơ-ron sâu hoạt động như một quy trình chưng cất thông tin nhiều giai đoạn, nơi thông tin đi qua các bộ lọc kế tiếp và trở nên tinh khiết hơn (tức là hữu ích hơn đối với một tác vụ cụ thể).

- Cơ chế hoạt động cốt lõi:

- Deep Learning là một phép biến đổi hình học phức tạp (complex geometric transformation) trong không gian nhiều chiều, được thực hiện thông qua một chuỗi các bước đơn giản.
- Mô hình được tham số hóa bằng các trọng số (weights) và học cách điều chỉnh các trọng số này thông qua một tín hiệu phản hồi được gọi là hàm mất mát (loss function).

- Quá trình điều chỉnh các trọng số để giảm thiểu hàm mất mát được gọi là Gradient Descent (Hạ gradient). Thuật toán Backpropagation (Lan truyền ngược) là thuật toán trung tâm trong Deep Learning, cho phép tính toán gradient này một cách hiệu quả.
- Deep Learning làm cho các tác vụ khó khăn (như phân loại hình ảnh, nhận dạng giọng nói) trở nên khả thi bằng cách tự động tìm kiếm các biểu diễn dữ liệu hữu ích.

- So sánh Deep Learning và Machine Learning truyền thống

Đặc điểm so sánh	Deep Learning	Machine Learning truyền thống (Shallow Learning)
Kiến trúc (Độ sâu)	Sâu (Deep): Có nhiều lớp biểu diễn nối tiếp nhau (hàng chục đến hàng trăm lớp).	Nông (Shallow): Thường chỉ tập trung vào việc học một hoặc hai lớp biểu diễn.
Kỹ thuật tính năng (Feature Engineering)	Tự động hóa hoàn toàn: Mô hình tự động học tất cả các tính năng (feature) trong một lần chạy. Thay thế các quy trình phức tạp, dễ hỏng bằng một mô hình Deep Learning đơn giản, end-to-end.	Thủ công: Đòi hỏi con người phải dành nhiều công sức để tạo ra các tính năng hữu ích (manual feature engineering) để dữ liệu dễ xử lý hơn (ví dụ: histogram pixel).
Quá trình học tập	Học đồng thời (joint feature learning): Tất cả các lớp biểu diễn được học cùng lúc, cho phép học các biểu diễn phức tạp,	Học nối tiếp (Greedy stacking): Các ứng dụng nối tiếp của các phương pháp nông cho thấy lợi ích giảm nhanh chóng.

	trừu tượng thông qua chuỗi các phép biến đổi đơn giản.	
Hiệu suất & Ứng dụng	Vượt trội trong các tác vụ nhận thức (perceptual tasks) như thị giác máy tính (computer vision), nhận dạng giọng nói, và xử lý ngôn ngữ tự nhiên (NLP).	Phù hợp nhất với các bài toán có dữ liệu có cấu trúc (structured data) hoặc phi nhận thức.
Khả năng mở rộng (Scalability)	Rất phù hợp để song song hóa trên GPU hoặc TPU. Có thể được huấn luyện trên các bộ dữ liệu có kích thước tùy ý thông qua các batch nhỏ.	Khó mở rộng trên các bộ dữ liệu lớn và phức tạp (ví dụ: SVM gặp khó khăn với dữ liệu lớn).
Khả năng tái sử dụng (Reusability)	Các mô hình đã huấn luyện có thể tái sử dụng (repurpose) và điều chỉnh (fine-tuning) cho các bài toán khác nhau, mang lại lợi thế lớn cho các bài toán dữ liệu nhỏ.	Ít linh hoạt hơn trong việc tái sử dụng tính năng trên các bài toán khác biệt.

2. Giải thích vai trò của tensor trong deep learning. Tensor khác gì so với mảng NumPy?

- Về bản chất, tensor là một hộp chứa dữ liệu, thường là dữ liệu số. Tensor là sự khái quát hóa của ma trận (matrices) lên một số lượng chiều (dimension) tùy ý. Trong bối cảnh của tensor, một chiều thường được gọi là trục (axis).
- Các thuộc tính chính xác định một tensor bao gồm:

1. Số trục (Rank/ndim): Số lượng chiều của tensor. Ví dụ, ma trận là tensor hạng 2, và tensor hạng 3 có ba trục.
 2. Hình dạng (Shape): Một tuple các số nguyên mô tả kích thước của tensor dọc theo mỗi trục. Ví dụ, ma trận 3x5 có shape là (3, 5).
 3. Loại dữ liệu (Data type - dtype): Loại dữ liệu chứa trong tensor, chẳng hạn như float32, uint8, v.v..
- Trong Deep Learning, bạn thường thao tác với các tensor có hạng từ 0 đến 4, mặc dù có thể lên đến 5 nếu xử lý dữ liệu video.
- Rank-0 tensor (Scalar): Chứa một số duy nhất.
 - Rank-1 tensor (Vector): Một mảng số.
 - Rank-2 tensor (Matrix): Dữ liệu được sắp xếp theo hàng và cột.
- Chức năng cốt lõi trong Deep Learning: Trong Deep Learning, tensor đóng vai trò là "ngôn ngữ" và "vật liệu xây dựng" của mọi thứ:
- Biểu diễn dữ liệu: Dữ liệu đầu vào thô, dù là văn bản, hình ảnh hay chuỗi thời gian, đều phải được vector hóa (vectorized) và chuyển thành tensor trước khi đưa vào mạng nơ-ron. Ví dụ, ảnh MNIST ban đầu là tensor hạng 3 (60000, 28, 28).
 - Thực hiện tính toán: Tất cả các phép biến đổi được học bởi mạng nơ-ron sâu đều có thể được quy giản thành một số ít phép toán tensor (tensor operations) cơ bản (như cộng tensor, nhân tensor, tích vô hướng - dot product) được áp dụng lên các tensor dữ liệu số.
 - Lưu trữ kiến thức: Trọng số (weights) hoặc tham số có thể huấn luyện (trainable parameters) của các lớp trong mô hình cũng là tensor. Các trọng số này lưu trữ kiến thức mà mô hình học được thông qua quá trình huấn luyện.

- Giải thích hình học: Toàn bộ quá trình Deep Learning có thể được giải thích một cách hình học, nơi mọi thứ là vector (tức là một điểm trong không gian hình học).

- So sánh Tensor và Mảng NumPy:

Đặc điểm	TensorFlow Tensor	NumPy Array
Tính thay đổi (Mutability)	Không thể gán giá trị (non-assignable). Chúng là hằng số.	Có thể gán giá trị (assignable).
Quản lý trạng thái	Sử dụng lớp chuyên biệt tf.Variable để quản lý trạng thái có thể thay đổi (ví dụ: trọng số của mô hình).	Không có lớp chuyên biệt này, dữ liệu được thay đổi trực tiếp.
Tính toán Gradient	Hỗ trợ tự động tính toán gradient thông qua API GradientTape. Tính năng này là cốt lõi cho việc học (training) mô hình DL.	Không có tính năng tự động tính toán gradient tích hợp.
Tăng tốc phần cứng	Có thể chạy trên GPU và TPU (Bộ xử lý Tensor).	Chủ yếu chạy trên CPU, mặc dù các phép toán thường được tối ưu hóa thông qua các thư viện cấp thấp (BLAS) được triển khai bằng Fortran hoặc C.
Triển khai (Deployment)	Có thể dễ dàng phân phối tính toán trên nhiều máy và xuất chương trình sang các runtime	Phù hợp cho tính toán cục bộ và phát triển mô hình ban đầu.

	khác (như C++, JavaScript, TensorFlow Lite).	
--	--	--

3. Mô tả lifecycle của một mô hình deep learning trong Keras (chuẩn bị dữ liệu → xây dựng mô hình → compile → train → evaluate → inference).

- Chuẩn bị Dữ liệu (Data Preparation)

Mọi dữ liệu đầu vào cho các mô hình Deep Learning đều phải được vector hóa (vectorized) và biểu diễn dưới dạng các tensor số.

- Vector hóa: Đây là quá trình chuyển đổi dữ liệu thô (như văn bản, hình ảnh, hoặc chuỗi thời gian) thành các mảng số (tensors).
- Tiền xử lý (Preprocessing): Dữ liệu thường cần được điều chỉnh. Đối với dữ liệu có các tính năng (features) nằm trong các phạm vi khác nhau, mỗi tính năng cần được chia tỷ lệ độc lập (scale independently). Ví dụ, giá trị pixel của hình ảnh (thường nằm trong khoảng) thường được chia tỷ lệ về khoảng.
- Định dạng Tensor: Dữ liệu phải được định hình (reshape) để phù hợp với kiến trúc mạng. Ví dụ, dữ liệu hình ảnh cho mạng tích chập (convnets) thường có hình dạng (image_height, image_width, image_channels).
- Tách tập dữ liệu: Chia dữ liệu thành các tập huấn luyện (training), xác thực (validation), và kiểm thử (test) là thực hành tiêu chuẩn để đánh giá mô hình một cách đáng tin cậy.

- Xây dựng Mô hình (Model Building)

Xây dựng mô hình là việc xác định kiến trúc mạng nơ-ron.

- Layer (Lớp): Layer là khối xây dựng cốt lõi của mạng nơ-ron, là một mô-đun xử lý dữ liệu nhận một hoặc nhiều tensor làm đầu vào và trả về các tensor đầu ra. Các layer học cách trích xuất các biểu diễn ngày càng có ý nghĩa từ dữ liệu đầu vào.

- Trọng số (Weights): Các layer thường có trạng thái: đó là trọng số (weights) hoặc tham số có thể huấn luyện (trainable parameters) là các tensor chứa kiến thức mà mô hình học được trong quá trình huấn luyện.
- API trong Keras: Keras cung cấp nhiều cách để xây dựng mô hình:
 - Sequential model: Xây dựng mô hình bằng cách xếp chồng các layer theo trình tự đơn giản.
 - Functional API: Phương pháp phổ biến nhất, cho phép tạo ra các kiến trúc phức tạp hơn dưới dạng đồ thị các layer (graph of layers).
 - Model Subclassing: Tùy chọn cấp thấp cho phép kiểm soát hoàn toàn thông qua việc kế thừa lớp Model.

- Compile (Cấu hình quá trình học)

Bước compile() được sử dụng để cấu hình quy trình học tập của mô hình. Nó yêu cầu ba yếu tố chính:

1. Hàm mất mát (Loss function): Đây là đại lượng mà mô hình sẽ cố gắng giảm thiểu trong quá trình huấn luyện. Hàm mất mát phải là một thước đo thành công cho tác vụ đang giải quyết. Ví dụ:
 - sparse_categorical_crossentropy cho bài toán phân loại đa lớp với nhãn là số nguyên.
 - binary_crossentropy cho phân loại nhị phân (binary classification).
 - mean_squared_error (mse) cho các bài toán hồi quy (regression).
2. Bộ tối ưu hóa (Optimizer): Là cơ chế mà mô hình tự cập nhật dựa trên dữ liệu huấn luyện, nhằm cải thiện hiệu suất. Bộ tối ưu hóa thực hiện một biến thể cụ thể của Thuật toán Hạ Gradient ngẫu nhiên theo lô nhỏ (mini-batch Stochastic Gradient Descent - SGD). Ví dụ phổ biến: rmsprop, adam.
3. Chỉ số (Metrics): Là các thước đo thành công bạn muốn theo dõi trong quá trình huấn luyện và xác thực (ví dụ: accuracy - độ chính xác, mae - lỗi tuyệt

đối trung bình). Không giống như hàm mất mát, quá trình huấn luyện không trực tiếp tối ưu hóa các chỉ số này.

- Train (Huấn luyện)

Quá trình huấn luyện được thực hiện bằng cách gọi phương thức `fit()` của mô hình.

- Vòng lặp huấn luyện: Phương thức `fit()` triển khai vòng lặp huấn luyện, hoạt động bằng cách lặp lại các bước sau cho đến khi hàm mất mát đủ thấp:
 - Lấy một lô dữ liệu nhỏ (batch) gồm các mẫu huấn luyện và mục tiêu tương ứng.
 - Chạy mô hình trên lô dữ liệu (Forward Pass) để lấy dự đoán.
 - Tính toán giá trị mất mát (loss).
 - Tính toán gradient của hàm mất mát đối với các tham số của mô hình (Backward Pass).
 - Cập nhật tất cả các trọng số theo hướng ngược lại với gradient để giảm thiểu mất mát, sử dụng bộ tối ưu hóa đã cấu hình. Quá trình này được gọi là Backpropagation.
- Epochs: Là số lần mô hình lặp lại trên toàn bộ dữ liệu huấn luyện.
- Callbacks: Có thể sử dụng các đối tượng Callback (ví dụ: `EarlyStopping`, `ModelCheckpoint`) để lên lịch các hành động được thực hiện tại các điểm cụ thể trong quá trình huấn luyện, chẳng hạn như dừng sớm để ngăn ngừa overfitting.
- Validation Data: Có thể truyền dữ liệu xác thực thông qua đối số `validation_data` trong `fit()` để theo dõi hiệu suất của mô hình trên dữ liệu chưa được nhìn thấy trong quá trình huấn luyện.

- Evaluate (Đánh giá)

Sau khi huấn luyện, hiệu suất của mô hình cần được đánh giá trên tập dữ liệu kiểm thử (test set) hoàn toàn mới.

- API: Sử dụng phương thức `model.evaluate()`.
- Mục đích: `evaluate()` tính toán giá trị mất mát và các chỉ số đã được cấu hình trên dữ liệu kiểm thử, cung cấp một ước tính tin cậy về khả năng khái quát hóa (generalization power) của mô hình (khả năng hoạt động chính xác trên đầu vào chưa từng thấy).

- Inference (Dự đoán)

Giai đoạn cuối cùng là sử dụng mô hình đã huấn luyện để tạo ra dự đoán trên dữ liệu mới.

- API: Sử dụng phương thức `model.predict()`.
- Đầu ra: Phương thức này nhận đầu vào (thường là mảng NumPy hoặc Dataset) và trả về các dự đoán dưới dạng mảng NumPy. Quá trình này được gọi là suy luận (Inference).

4. Gradient descent hoạt động như thế nào? Batch GD, mini-batch GD, stochastic GD khác nhau ra sao?

- Gradient Descent (GD), hay Hạ Gradient, là cơ chế tối ưu hóa (optimization mechanism) điều khiển quá trình học tập của các mạng nơ-ron hiện đại. Nó là phương pháp chính được sử dụng để tìm ra các giá trị trọng số (weights) (hay tham số có thể huấn luyện) của mô hình nhằm giảm thiểu hàm mất mát (loss function).

- Vai trò của Trọng số và Hàm mất mát: Ban đầu, các ma trận trọng số (weights) của các lớp trong mạng nơ-ron thường được điền bằng các giá trị ngẫu nhiên nhỏ (khởi tạo ngẫu nhiên). Sau đó, mô hình cần điều chỉnh dần các trọng số này dựa trên một tín hiệu phản hồi. Tín hiệu phản hồi đó chính là hàm mất mát, đo lường sự không khớp (mismatch) giữa dự đoán của mô hình (y_{pred}) và mục tiêu mong muốn (y_{true}) trên dữ liệu huấn luyện.

- Vòng lặp Huấn luyện cốt lõi: Quá trình điều chỉnh các trọng số được gọi là huấn luyện (training) hoặc học (learning). Vòng lặp huấn luyện lặp đi lặp lại các bước sau cho đến khi giá trị mất mát đủ thấp:

1. Lấy một lô (batch) mẫu huấn luyện (x) và mục tiêu tương ứng (y_{true}).
2. Chạy mô hình (Forward Pass): Chạy mô hình trên x để có được dự đoán (y_{pred}).
3. Tính toán mất mát: Tính toán giá trị mất mát (loss value), đo lường sự khác biệt giữa y_{pred} và y_{true} .
4. Tính toán Gradient (Backward Pass): Tính toán gradient của hàm mất mát đối với các tham số (trọng số) của mô hình. Gradient mô tả cách giá trị mất mát thay đổi khi các hệ số của mô hình thay đổi theo các hướng khác nhau.
5. Cập nhật Trọng số: Cập nhật tất cả các trọng số của mô hình theo hướng ngược lại với gradient, nhằm giảm thiểu giá trị mất mát trên lô dữ liệu đó.

- Tốc độ học (Learning Rate): Mức độ lớn của bước cập nhật trọng số (tức là tốc độ mô hình học) được xác định bởi một yếu tố tỷ lệ nhỏ gọi là tốc độ học ($learning_rate$). Việc chọn tốc độ học phù hợp là rất quan trọng: nếu quá nhỏ, quá trình sẽ mất nhiều lần lặp; nếu quá lớn, các cập nhật có thể đưa mô hình đến các vị trí ngẫu nhiên trên bề mặt mất mát.

- So sánh Batch GD, mini-batch GD và Stochastic GD

Biến thể	Cỡ lô (Batch Size)	Độ chính xác Gradient	Chi phí tính toán	Ghi chú
Batch Gradient Descent (Batch GD)	Toàn bộ dữ liệu huấn luyện	Rất cao, chính xác	Rất đắt, chậm	Hiếm khi được sử dụng trong Deep Learning thực tế do chi phí quá cao.

Stochastic Gradient Descent (SGD) (True SGD)	Chỉ một mẫu dữ liệu (single sample)	Thấp, nhiều (noisy)	Rẻ nhất, nhanh nhất mỗi bước	Thuật ngữ "stochastic" (ngẫu nhiên) đề cập đến việc lô dữ liệu (chỉ 1 mẫu) được rút ra một cách ngẫu nhiên.
Mini-batch SGD (Mini-batch GD)	Một lô mẫu nhỏ (thường 8 đến 128)	Cân bằng, ít nhiều hơn SGD	Cân bằng, hiệu quả	Biến thể phổ biến và hiệu quả nhất được sử dụng trong thực tế.

5. Mục đích của hàm kích hoạt (activation function). So sánh ReLU, sigmoid, tanh.

- Mục đích của Hàm Kích Hoạt: Nếu một lớp Dense không sử dụng hàm kích hoạt, nó chỉ thực hiện hai phép toán tuyến tính: tích vô hướng (dot product) với ma trận trọng số W và phép cộng vector với vector độ lệch b . Kết quả là một phép biến đổi affine (affine transform), tương đương với phép biến đổi tuyến tính.

- Mục đích chính của hàm kích hoạt (non-linearity) là:

1. Mở rộng Không gian Giả thuyết (Hypothesis Space): Nếu một mạng nơ-ron sâu được xây dựng hoàn toàn từ các lớp Dense tuyến tính (không có hàm kích hoạt), chuỗi các phép toán đó sẽ tương đương với một lớp Dense duy nhất. Nói cách khác, một mạng "sâu" như vậy sẽ chỉ là một mô hình tuyến tính trá hình.
2. Thực hiện các Biến đổi Hình học Phức tạp: Để mạng nơ-ron sâu có thể học các biểu diễn phong phú, phức tạp và có chiều sâu (deep representations), nó cần phải có khả năng thực hiện các biến đổi hình học phi tuyến tính phức tạp (complex, non-linear geometric transformations).

3. Cho phép Kiến trúc Sâu: Nhờ có các hàm kích hoạt như ReLU, một chuỗi các lớp Dense mới có thể triển khai các phép biến đổi hình học phi tuyến tính rất phức tạp, mang lại không gian giả thuyết phong phú hơn cho mạng nơ-ron sâu của bạn.

- So sánh ReLU, Sigmoid và Tanh

Hàm kích hoạt	Công thức và Đặc điểm	Phạm vi Giá trị Đầu ra	Ứng dụng tiêu biểu
ReLU (Rectified Linear Unit)	$\text{relu}(x) = \max(x, 0)$. Chức năng là đưa các giá trị âm về 0.	[0, vô cùng)	Phổ biến nhất trong các lớp ẩn (intermediate layers) của mạng Deep Learning.
Sigmoid	Hàm "bóp" (squashes) các giá trị tùy ý thành phân phối xác suất.		Lớp cuối cùng của các bài toán phân loại nhị phân (binary classification), để dự đoán xác suất (điểm số giữa 0 và 1).
Tanh (Hyperbolic Tangent)	Được sử dụng để thêm phi tuyến tính vào quá trình biến đổi của mạng nơ-ron hồi quy (RNN).	[-1, 1]	Được sử dụng trong tế bào RNN đơn giản (SimpleRNN) để thêm phi tuyến tính trong quá trình kết hợp đầu vào và trạng thái hiện tại.

- ReLU: Là hàm kích hoạt phổ biến nhất, được sử dụng trong hầu hết các mạng Deep Learning. Nó giúp mạng học các biểu diễn phi tuyến tính bằng cách đưa các giá trị âm về 0, nhưng vẫn giữ nguyên các giá trị dương.

- Sigmoid: Hàm Sigmoid hữu ích vì nó "bóp" các giá trị đầu vào tùy ý vào khoảng $[0, 1]$. Điều này làm cho đầu ra dễ dàng được diễn giải là xác suất (ví dụ: xác suất một bài đánh giá là tích cực trong bài toán phân loại nhị phân).
- Tanh: Trong kiến trúc mạng nơ-ron hồi quy (RNN), hàm Tanh được sử dụng để áp dụng phi tuyến tính khi kết hợp đầu vào hiện tại với trạng thái trước đó để tính toán đầu ra mới.

6. Vì sao normalization và standardization dữ liệu lại quan trọng đối với deep learning?

- Mục đích cốt lõi của việc tiền xử lý dữ liệu

A. Đảm bảo tính đồng nhất (Homogeneity)

- Nếu dữ liệu đầu vào chứa các giá trị với phạm vi khác nhau—ví dụ, một tính năng dao động từ 0 đến 1, trong khi tính năng khác dao động từ 0 đến 1000—việc huấn luyện mô hình sẽ trở nên khó khăn hơn đáng kể.
- Mặc dù mô hình có thể tự thích nghi với dữ liệu không đồng nhất, nhưng việc chuẩn hóa sẽ giúp quá trình học tập dễ dàng hơn.
- Nói chung, hầu hết các giá trị nên nằm trong phạm vi nhỏ, điển hình là khoảng $[0, 1]$. Ví dụ, trong bài toán phân loại chữ số MNIST, dữ liệu ảnh (có giá trị pixel từ 0 đến 255) được chuyển thành float32 và chia cho 255 để nằm trong phạm vi $[0, 1]$ trước khi đưa vào mạng nơ-ron.

B. Cải thiện Tối ưu hóa (Optimization)

- Khi các tính năng có phạm vi khác nhau, bề mặt hàm mất mát (loss function surface) trong không gian tham số sẽ có hình dạng kéo dài và hẹp, khiến bộ tối ưu hóa (optimizer) phải thực hiện các bước cập nhật trọng số theo hình zigzag và mất nhiều thời gian để hội tụ.

- Khi dữ liệu được chuẩn hóa, bề mặt mất mát trở nên trơn trịa hơn, cho phép bộ tối ưu hóa tìm ra điểm tối ưu một cách nhanh chóng và ổn định hơn.
- Kỹ thuật chuẩn hóa cụ thể: Khi các tính năng trong dữ liệu đầu vào có các phạm vi giá trị khác nhau, phương pháp thực hành tốt nhất được áp dụng rộng rãi là chuẩn hóa tính năng (feature-wise normalization):

1. Trừ giá trị trung bình: Trừ đi giá trị trung bình (mean) của tính năng đó, làm cho tính năng tập trung quanh 0.
2. Chia cho độ lệch chuẩn: Chia cho độ lệch chuẩn (standard deviation), giúp tính năng đó có độ lệch chuẩn đơn vị (1).

Quá trình này được thực hiện độc lập cho từng tính năng.

- Lưu ý quan trọng về dữ liệu kiểm thử (Test Data):

- Các đại lượng (trung bình và độ lệch chuẩn) được sử dụng để chuẩn hóa dữ liệu kiểm thử phải được tính toán chỉ dựa trên dữ liệu huấn luyện (training data).
- Bạn không bao giờ được sử dụng bất kỳ đại lượng nào được tính toán trên dữ liệu kiểm thử trong quy trình làm việc của mình, ngay cả đối với các tác vụ đơn giản như chuẩn hóa dữ liệu.

- Tầm quan trọng của Normalization bên trong mạng: Ngoài việc chuẩn hóa dữ liệu đầu vào, khái niệm chuẩn hóa còn được mở rộng vào bên trong kiến trúc mạng nơ-ron thông qua Batch Normalization (Chuẩn hóa Batch).

- Mục đích của chuẩn hóa (dù là ở lớp đầu vào hay lớp trung gian) là làm cho các mẫu dữ liệu mà mô hình nhìn thấy giống nhau hơn, giúp mô hình học và khái quát hóa (generalize) tốt hơn trên dữ liệu mới.
- Mặc dù dữ liệu đầu vào có thể đã được chuẩn hóa (ví dụ: có giá trị trung bình bằng 0 và phương sai đơn vị), đầu ra của các lớp Dense hoặc Conv2D tiếp theo có thể không giữ được tính chất này.

- Việc áp dụng Batch Normalization ở các lớp trung gian giúp cải thiện sự lan truyền gradient (gradient propagation) và cho phép các kiến trúc mạng nơ-ron trở nên sâu hơn (deeper) mà vẫn có thể huấn luyện được.

7. Overfitting là gì? Liệt kê các kỹ thuật chống overfitting trong sách (dropout, giảm số chiều, L1/L2 regularization, data augmentation, early stopping...).

- Overfitting là một vấn đề trung tâm trong Machine Learning. Nó mô tả thực tế rằng các mô hình học máy có xu hướng hoạt động kém hơn trên dữ liệu mới (chưa từng thấy) so với trên dữ liệu huấn luyện của chúng. Mục tiêu cơ bản của học máy là đạt được khả năng khái quát hóa (generalization) tốt, tức là khả năng hoạt động tốt trên các điểm dữ liệu mà mô hình chưa từng gặp trước đây.

- Mối quan hệ giữa Tối ưu hóa và Khái quát hóa Overfitting xảy ra do sự căng thẳng giữa Tối ưu hóa và Khái quát hóa:

1. Tối ưu hóa (Optimization): Quá trình điều chỉnh mô hình để đạt hiệu suất tốt nhất có thể trên dữ liệu huấn luyện.
2. Khái quát hóa (Generalization): Mức độ mô hình đã huấn luyện hoạt động tốt trên dữ liệu mới.

- Biểu hiện và Nguyên nhân: Trong quá trình huấn luyện, hiệu suất trên dữ liệu xác thực (validation data) sẽ tăng lên và sau đó không thể tránh khỏi việc đạt đỉnh rồi bắt đầu suy giảm. Khi đó, mô hình đang bắt đầu quá khớp:

- Mô hình bắt đầu học các khuôn mẫu dành riêng cho dữ liệu huấn luyện nhưng lại gây hiểu lầm hoặc không liên quan khi áp dụng cho dữ liệu mới.
- Overfitting đặc biệt dễ xảy ra khi dữ liệu nhiễu (chứa các điểm ngoại lai hoặc được gán nhãn sai), hoặc khi dữ liệu chứa các tính năng hiếm hoặc

tương quan giả (spurious correlations) mà mô hình học được, nhưng chúng không khái quát hóa được.

Ví dụ, đối với bài toán phân loại đánh giá phim IMDB, nếu mô hình được huấn luyện quá 4 epoch, nó sẽ quá tối ưu hóa trên dữ liệu huấn luyện và thu được kết quả tệ hơn trên dữ liệu xác thực.

- Các kỹ thuật chống Overfitting (Regularization) Quá trình chống lại overfitting được gọi là điều chuẩn hóa (regularization). Điều chuẩn hóa là một tập hợp các phương pháp thực hành tốt nhất nhằm ngăn cản tích cực khả năng khớp hoàn hảo của mô hình với dữ liệu huấn luyện. Các kỹ thuật điều chuẩn này giúp mô hình đơn giản hơn, mượt mà hơn và ít đặc trưng hóa với tập huấn luyện hơn.

1. Giảm kích thước Mạng (Reducing the Network's Size)

Giảm kích thước mô hình (số lượng tham số có thể học) là cách đơn giản nhất để giảm nhẹ overfitting.

- Nếu mô hình có nguồn lực ghi nhớ hạn chế (ít tham số), nó sẽ buộc phải tập trung vào việc học các biểu diễn nén có sức mạnh dự đoán tốt hơn, những biểu diễn có khả năng khái quát hóa tốt hơn.
- Khi có ít dữ liệu huấn luyện, việc sử dụng một mô hình nhỏ (chỉ một hoặc hai lớp trung gian) là điều cần thiết để tránh quá khớp nghiêm trọng.

2. Thêm Dropout

Dropout là một trong những kỹ thuật điều chuẩn hiệu quả và được sử dụng phổ biến nhất cho mạng nơ-ron.

- Dropout, khi áp dụng cho một lớp, bao gồm việc ngẫu nhiên loại bỏ (đặt về 0) một số đặc trưng đầu ra của lớp đó trong quá trình huấn luyện. Tỷ lệ dropout thường được đặt trong khoảng 0.2 đến 0.5.

- Mục tiêu là phá vỡ các khuôn mẫu ngẫu nhiên (happenstance patterns) hoặc các tương quan giả. Bằng cách đưa nhiễu vào giá trị đầu ra của lớp, nó ngăn mô hình ghi nhớ các mẫu không có ý nghĩa.
- Lưu ý rằng, trong quá trình dự đoán (inference), không có đơn vị nào bị loại bỏ; thay vào đó, các giá trị đầu ra của lớp được chia tỷ lệ theo hệ số bằng tỷ lệ dropout để cân bằng.

3. Điều chuẩn Trọng số L1 và L2 (L1/L2 Regularization)

Điều chuẩn trọng số áp dụng nguyên tắc Dao cạo của Occam (Occam's razor): mô hình đơn giản hơn ít có khả năng quá khớp hơn.

Điều chuẩn trọng số được thực hiện bằng cách thêm vào hàm mất mát một chi phí liên quan đến việc có trọng số lớn. Điều này buộc các trọng số chỉ nhận các giá trị nhỏ, làm cho phân phối giá trị trọng số trở nên "thường xuyên" (regular) hơn.

- L1 regularization: Chi phí được thêm vào tỷ lệ thuận với giá trị tuyệt đối của các hệ số trọng số (chuẩn L1).
- L2 regularization (còn gọi là Weight Decay - Suy giảm trọng số): Chi phí được thêm vào tỷ lệ thuận với bình phương của giá trị hệ số trọng số (chuẩn L2).

Kỹ thuật này thường được ưa chuộng hơn đối với các mô hình Deep Learning nhỏ hơn.

4. Sử dụng Dừng sớm (Early Stopping)

Dừng sớm là một kỹ thuật hiệu quả để cải thiện khả năng khái quát hóa.

- Kỹ thuật này bao gồm việc gián đoạn quá trình huấn luyện rất lâu trước khi đạt đến mức mất mát huấn luyện thấp nhất có thể.
- Bạn có thể tìm thấy điểm chính xác trong quá trình huấn luyện mà bạn đạt được độ khớp khái quát hóa nhất.

- Trong Keras, điều này thường được thực hiện bằng cách sử dụng callback EarlyStopping, sẽ tự động ngắt quá trình huấn luyện ngay khi các chỉ số xác thực ngừng cải thiện.

5. Tăng cường dữ liệu (Data Augmentation)

Tăng cường dữ liệu là một kỹ thuật mạnh mẽ, đặc trưng cho thị giác máy tính, được sử dụng để giảm nhẹ overfitting.

- Quá khớp xảy ra do có quá ít mẫu để học, khiến mô hình không thể khái quát hóa. Data Augmentation giải quyết vấn đề này bằng cách tạo ra nhiều dữ liệu huấn luyện hơn từ các mẫu hiện có thông qua các phép biến đổi ngẫu nhiên.
- Mục tiêu là mô hình sẽ không bao giờ thấy cùng một hình ảnh chính xác hai lần trong quá trình huấn luyện, giúp mô hình tiếp xúc với nhiều khía cạnh hơn của dữ liệu để nó có thể khái quát hóa tốt hơn.
- Ví dụ về các phép biến đổi bao gồm lật ngang ngẫu nhiên (RandomFlip), xoay ngẫu nhiên (RandomRotation), hoặc phóng to/thu nhỏ ngẫu nhiên (RandomZoom).

6. Giảm số chiều (Feature Selection)

Mặc dù tài liệu không liệt kê rõ "giảm số chiều" là một kỹ thuật chống overfitting riêng biệt, nhưng nó liên quan đến các kỹ thuật khác:

- Lựa chọn tính năng (Feature Selection): Nếu không chắc chắn liệu các tính năng có cung cấp thông tin hay không, bạn nên thực hiện lựa chọn tính năng trước khi huấn luyện. Hạn chế dữ liệu IMDB chỉ với 10.000 từ phổ biến nhất là một hình thức lựa chọn tính năng thô sơ. Kỹ thuật này giúp lọc bỏ các kênh nhiễu trắng, thứ dẫn đến quá khớp.

- Vector hóa/Tiền xử lý: Hạn chế từ vựng (ví dụ: chỉ giữ lại 20.000 từ) là một bước tiền xử lý chuẩn hóa giúp tránh một không gian đặc trưng quá lớn, nơi hầu hết các tính năng không có nội dung thông tin và dễ dẫn đến quá khớp.

8. Ý nghĩa của hàm loss và optimizer trong mô hình. Nêu ví dụ loss phù hợp cho classification và regression.

- Hàm mất mát (còn được gọi là hàm mục tiêu - *objective function* hoặc hàm chi phí - *cost function*) là đại lượng mà mô hình sẽ cố gắng giảm thiểu trong quá trình huấn luyện.

1. Đo lường Chất lượng Đầu ra: Hàm mất mát nhận đầu vào là dự đoán của mạng Y' và mục tiêu thực Y_{true} để tính toán một điểm số khoảng cách (distance score). Điểm số này đo lường mức độ khớp (hoặc không khớp) giữa đầu ra hiện tại của mô hình và đầu ra mong đợi.
2. Tín hiệu Phản hồi (Feedback Signal): Điểm mất mát (loss score) này được sử dụng làm tín hiệu phản hồi để điều chỉnh giá trị của các trọng số (weights).
3. Thước đo Thành công: Hàm mất mát phải là thước đo thành công cho nhiệm vụ mà mô hình đang cố gắng giải quyết. Việc chọn hàm mất mát phù hợp là cực kỳ quan trọng, bởi vì mạng nơ-ron sẽ tìm mọi cách để giảm thiểu nó, do đó mục tiêu này phải hoàn toàn tương quan với thành công của nhiệm vụ.

- Ý nghĩa của Bộ Tối ưu hóa (Optimizer)

Bộ tối ưu hóa là cơ chế thông qua đó mô hình sẽ tự cập nhật dựa trên dữ liệu huấn luyện để cải thiện hiệu suất.

1. Điều chỉnh Trọng số: Công việc của bộ tối ưu hóa là sử dụng điểm số mất mát để điều chỉnh giá trị của các trọng số.

2. Triển khai Hạ Gradient: Bộ tối ưu hóa thực hiện một biến thể cụ thể của thuật toán Hạ Gradient ngẫu nhiên theo lô nhỏ (mini-batch Stochastic Gradient Descent - SGD).
3. Sử dụng Gradient: Bộ tối ưu hóa chỉ định chính xác cách thức gradient của hàm mất mát sẽ được sử dụng để cập nhật các tham số (trọng số). Nó triển khai thuật toán Backpropagation (Lan truyền ngược) để tính toán gradient và cập nhật trọng số.

Các bộ tối ưu hóa phổ biến bao gồm SGD (có hoặc không có momentum), RMSprop và Adam.

- Ví dụ Hàm Loss phù hợp cho Classification và Regression

A. Classification (Bài toán Phân loại)

Loại Phân loại	Hàm kích hoạt lớp cuối	Hàm mất mát (Loss Function)	Ghi chú
Phân loại Nhị phân (Binary Classification)	Sigmoid (một đơn vị)	Binary Crossentropy	Phù hợp khi mô hình đầu ra là một xác suất (giữa 0 và 1).
Phân loại Đa lớp, Đơn nhãn (Multiclass, Single-label)	Softmax (N đơn vị)	Categorical Crossentropy	Yêu cầu nhãn phải được mã hóa dạng one-hot encoding.
Phân loại Đa lớp, Đơn nhãn (Alternative)	Softmax (N đơn vị)	Sparse Categorical Crossentropy	Sử dụng khi nhãn được mã hóa dưới dạng số nguyên (integers) thay vì one-hot encoding.

B. Regression (Bài toán Hồi quy)

Loại Hồi quy	Hàm kích hoạt lớp cuối	Hàm mất mát (Loss Function)	Ghi chú
Hồi quy Vô hướng (Scalar Regression)	Không có hàm kích hoạt	Mean Squared Error (MSE)	MSE (Bình phương trung bình của lỗi) là hàm mất mát được sử dụng rộng rãi nhất cho hồi quy. Nó thường được ưu tiên hơn vì nó mượt mà (smooth) quanh điểm 0, giúp quá trình hạ gradient dễ dàng hơn.

- Thước đo Đánh giá (Metrics): Mặc dù MSE là hàm mất mát thường được sử dụng cho hồi quy, một chỉ số phổ biến để theo dõi (nhưng không trực tiếp tối ưu hóa) là Mean Absolute Error (MAE) (Sai số tuyệt đối trung bình).

9. Mục đích của validation set. Phân biệt training–validation–test.

- Mục đích của Validation Set (Tập Xác thực): Mục đích chính của tập xác thực là theo dõi khả năng khái quát hóa (generalization power) của mô hình trong quá trình phát triển và điều chỉnh siêu tham số (hyperparameters) của mô hình mà không làm "rò rỉ thông tin" (information leak) từ tập kiểm thử (test set).

A. Ngăn ngừa Overfitting trong quá trình phát triển: Phát triển một mô hình Deep Learning luôn liên quan đến việc điều chỉnh cấu hình của nó—ví dụ, chọn số lượng lớp (layers) hoặc kích thước của các lớp (được gọi là siêu tham số của mô hình).

- Tập dữ liệu xác thực được sử dụng làm tín hiệu phản hồi để điều chỉnh siêu tham số này.

- Quá trình điều chỉnh cấu hình mô hình dựa trên hiệu suất của nó trên tập xác thực là một hình thức học tập (learning), một quá trình tìm kiếm cấu hình tốt trong không gian tham số.
- Nếu không có Validation Set, người ta sẽ buộc phải sử dụng Test Set để điều chỉnh siêu tham số. Tuy nhiên, nếu bạn lặp lại quá trình này nhiều lần—chạy thử nghiệm, đánh giá trên tập kiểm thử, và sửa đổi mô hình—bạn sẽ làm rò rỉ một lượng thông tin đáng kể về tập kiểm thử vào mô hình.
- Sự rò rỉ thông tin này dẫn đến việc mô hình hoạt động tốt một cách giả tạo trên dữ liệu kiểm thử, khiến cho thước đo khái quát hóa bị sai lệch.
- Do đó, Validation Set được sử dụng để đánh giá mô hình trong quá trình huấn luyện (chẳng hạn như theo dõi val_accuracy và val_loss) để xác định khi nào mô hình bắt đầu quá khớp (overfit) và cần dừng lại, ví dụ như sử dụng validation_data trong phương thức fit() của Keras.

B. Chọn Kỷ nguyên Tốt nhất (Best Epoch): Tập xác thực giúp tìm ra điểm tối ưu giữa Underfitting (Chưa khớp) và Overfitting (Quá khớp).

- Hiệu suất trên dữ liệu xác thực thường tăng lên và sau đó đạt đỉnh rồi bắt đầu suy giảm (overfitting).
- Bằng cách theo dõi các chỉ số xác thực (validation metrics) như val_loss hoặc val_accuracy, bạn có thể tìm thấy thời điểm chính xác nên dừng huấn luyện (ví dụ: sau khoảng 4 epoch trong ví dụ phân loại đánh giá phim IMDB). Kỹ thuật này thường được thực hiện bằng cách sử dụng callback EarlyStopping trong Keras.

- Phân biệt Training Set, Validation Set và Test Set

Tập dữ liệu	Mục đích	Cách sử dụng	Vai trò trong quá trình học
-------------	----------	--------------	-----------------------------

Training Set (Tập Huấn luyện)	Dữ liệu mô hình học từ đó	Dùng để chạy thuật toán Hạ Gradient (cập nhật trọng số).	Tối ưu hóa mô hình (Optimization).
Validation Set (Tập Xác thực)	Đánh giá hiệu suất của mô hình trong quá trình phát triển.	Dùng làm tín hiệu phản hồi để điều chỉnh siêu tham số (ví dụ: số epoch, kích thước lớp).	Kiểm soát Quá khớp và Khái quát hóa (Generalization).
Test Set (Tập Kiểm thử)	Đánh giá cuối cùng, độc lập về khả năng khái quát hóa của mô hình.	Chỉ được sử dụng một lần, sau khi mô hình đã sẵn sàng để triển khai (production).	Xác nhận cuối cùng về khả năng khái quát hóa.

- Các nguyên tắc quan trọng:

1. Tính riêng biệt (Disjoint): Tập huấn luyện và tập xác thực phải được tách biệt nghiêm ngặt.
2. Đánh giá cuối cùng: Tập kiểm thử được giữ hoàn toàn tách biệt, không có bất kỳ thông tin nào về nó được sử dụng trong quá trình huấn luyện hoặc điều chỉnh siêu tham số. Điều này đảm bảo rằng phép đo khả năng khái quát hóa là đáng tin cậy.
3. Tính đại diện: Các tập dữ liệu phải đại diện cho dữ liệu thực tế (production data) mà mô hình sẽ gặp phải. Nếu bạn đang dự đoán tương lai (ví dụ: chứng khoán), dữ liệu kiểm thử phải là dữ liệu phía sau (posterior) dữ liệu huấn luyện (nguyên tắc *arrow of time* - mũi tên thời gian).

- Nếu có ít dữ liệu, việc chia đơn giản có thể không đáng tin cậy do các tập xác thực hoặc kiểm thử quá nhỏ để đại diện cho toàn bộ dữ liệu. Trong trường hợp này,

các giao thức đánh giá nâng cao như K-fold Validation hoặc Iterated K-fold Validation with Shuffling nên được sử dụng.

10. Mô tả kiến trúc mạng fully connected (Dense). Ưu và nhược điểm.

- Mô tả Kiến trúc Mạng Fully Connected (Dense)

1. Công thức và Phép toán Tensor: Mô hình học được trong lớp Dense có thể được diễn giải dưới dạng ba phép toán tensor đơn giản:

$\text{output} = \text{activation}(\text{dot}(\text{input}, W) + b)$.

Trong đó:

- Đầu vào (input): Là một tensor (thường là ma trận hạng 2) có hình dạng (samples, features) (mẫu, đặc trưng).
- Tích vô hướng (dot(input, W)): Nhân ma trận đầu vào với ma trận trọng số (W) của lớp. W là một tensor chứa kiến thức của mô hình.
- Cộng vector (+ b): Cộng kết quả với vector độ lệch (b).
- Hàm kích hoạt (activation): Áp dụng một phép biến đổi phi tuyến tính (non-linear transformation) lên kết quả (ví dụ: relu hoặc sigmoid).

Lớp Dense không có hàm kích hoạt thực hiện một biến đổi affine ($\text{y} = W \cdot \text{x} + b$), là sự kết hợp của một biến đổi tuyến tính và một phép tịnh tiến.

2. Biểu diễn dữ liệu:

- Layer Dense được thiết kế để xử lý dữ liệu vector (vector data) được lưu trữ dưới dạng tensor hạng 2 có hình dạng (samples, features).
- Nó trích xuất các biểu diễn từ dữ liệu được đưa vào.
- Số đơn vị (units): Đối số đầu tiên truyền vào lớp Dense là số đơn vị trong lớp, tức là chiều (dimensionality) của không gian biểu diễn đầu ra của lớp đó.

3. Giả định kiến trúc:

- Mạng Dense không giả định bất kỳ cấu trúc cụ thể nào trong các đặc trưng đầu vào.
- Nó được gọi là *densely connected* vì các đơn vị của một lớp Dense được kết nối với mọi đơn vị khác. Lớp này cố gắng ánh xạ mối quan hệ giữa bất kỳ hai đặc trưng đầu vào nào.

- Ứng dụng phổ biến

- Phân loại nhị phân (Binary Classification): Thường kết thúc bằng một lớp Dense với một đơn vị và hàm kích hoạt sigmoid.
- Phân loại đa lớp, đơn nhãn (Multiclass, Single-label Classification): Kết thúc bằng một lớp Dense với số đơn vị bằng số lớp và hàm kích hoạt softmax.
- Hồi quy (Regression): Kết thúc bằng một lớp Dense với số đơn vị bằng số lượng giá trị cần dự đoán và không có hàm kích hoạt (purely linear layer).
- Giai đoạn cuối của các mạng phức tạp: Chúng thường được sử dụng làm giai đoạn phân loại hoặc hồi quy cuối cùng của hầu hết các mạng, ví dụ như sau các mạng tích chập (convnets) hoặc mạng hồi quy (RNNs).

2. Ưu điểm và Nhược điểm

Ưu điểm

1. Tính phổ quát và Đơn giản: Dense layer là khối xây dựng cơ bản, đơn giản và hiệu quả, có thể giải quyết được nhiều loại vấn đề.
2. Linh hoạt: Bằng cách điều chỉnh số lượng đơn vị và hàm kích hoạt, mạng Dense có thể được tùy chỉnh để xử lý các vấn đề phân loại (nhị phân, đa lớp) và hồi quy.

Nhược điểm và Hạn chế

1. Yêu cầu Phi tuyến tính (Non-linearity): Đây là hạn chế lớn nhất. Nếu một mạng nơ-ron sâu chỉ được tạo thành từ các lớp Dense không có hàm kích

hoạt, chuỗi các phép biến đổi affine đó sẽ tương đương với một lớp Dense duy nhất. Nói cách khác, một mạng sâu như vậy chỉ là một mô hình tuyến tính trá hình, và việc thêm nhiều lớp sẽ không mở rộng không gian giả thuyết (hypothesis space). Để có được không gian giả thuyết phong phú hơn và tận dụng các biểu diễn sâu, cần phải sử dụng hàm kích hoạt phi tuyến tính như ReLU.

2. Khả năng quá khớp (Overfitting) và Chi phí Tính toán:

- Tăng số lượng đơn vị (tăng chiều không gian biểu diễn) cho phép mô hình học các biểu diễn phức tạp hơn.
- Tuy nhiên, điều này cũng làm mô hình tốn kém hơn về mặt tính toán và dễ dẫn đến quá khớp (học các khuôn mẫu không mong muốn).

3. Không hiệu quả với dữ liệu có cấu trúc (Structured Data):

- Mạng Dense học các khuôn mẫu toàn cục (global patterns) trong không gian đặc trưng đầu vào.
- Đối với dữ liệu có cấu trúc cao như hình ảnh, mạng Dense kém hiệu quả hơn đáng kể so với các kiến trúc chuyên biệt như mạng tích chập (convnets), vốn học các khuôn mẫu cục bộ (local patterns) và mang tính bất biến dịch chuyển (translation-invariant).

11. Pooling layer (max/avg) và vai trò của nó.

- Max-Pooling (Gộp tối đa) và Average-Pooling (Gộp trung bình): Pooling Layer (Lớp gộp) hoạt động bằng cách trích xuất các cửa sổ (windows) từ bản đồ đặc trưng đầu vào (input feature maps). Thay vì áp dụng một phép biến đổi tuyến tính được học (như convolution kernel), lớp pooling thực hiện một phép biến đổi được mã hóa cứng (hardcoded tensor operation).

- Max-Pooling

- Cơ chế: Max-pooling hoạt động bằng cách trích xuất giá trị tối đa (max value) của mỗi kênh (channel) từ các cửa sổ cục bộ trên bản đồ đặc trưng.
- Tham số điển hình: Max-pooling thường được thực hiện với các cửa sổ 2×2 và bước nhảy (stride) là 2.

- Average-Pooling

- Cơ chế: Average-pooling (gộp trung bình) chuyển đổi từng mảng miếng cục bộ bằng cách lấy giá trị trung bình của mỗi kênh trên mảng miếng đó.
- So sánh với Max-Pooling: Max-pooling có xu hướng hoạt động tốt hơn. Max-pooling nhìn vào sự hiện diện tối đa của các đặc trưng (features) hơn là sự hiện diện trung bình của chúng, vì các đặc trưng thường mã hóa sự hiện diện không gian của một mẫu hoặc khái niệm.

- Vai trò và Mục đích của Pooling Layer

A. Giảm số lượng tham số và tính toán

Vai trò quan trọng nhất của việc giảm mẫu là để giảm số lượng hệ số bản đồ đặc trưng (feature-map coefficients) cần xử lý.

- Nếu không có các lớp pooling, kích thước bản đồ đặc trưng sẽ trở nên rất lớn. Ví dụ, trong một mô hình Convnet không có max-pooling, bản đồ đặc trưng cuối cùng có thể có $2 \times 2 \times 128 = 61,952$ hệ số trên mỗi mẫu.
- Khi làm phẳng (Flatten) một tensor có kích thước lớn như vậy để đưa vào lớp Dense (Fully Connected) cuối cùng, lớp Dense đó sẽ có hàng trăm nghìn tham số. Số lượng tham số quá lớn này làm mô hình trở nên quá khổ (far too large) và có thể dẫn đến quá khớp nghiêm trọng (intense overfitting).

B. Xây dựng Hệ thống cấp bậc Đặc trưng Không gian (Spatial Feature Hierarchies)

Pooling giúp tạo ra hệ thống cấp bậc đặc trưng không gian (spatial-filter hierarchies) bằng cách làm cho các lớp tích chập liên tiếp có thể nhìn thấy các cửa

số ngày càng lớn hơn (xét theo tỷ lệ phần trăm đầu vào ban đầu mà chúng bao phủ).

- Nếu không giảm mẫu, các khuôn mẫu cấp cao (high-level patterns) được học bởi lớp tích chập thứ ba sẽ vẫn rất nhỏ so với đầu vào ban đầu.
- Pooling đảm bảo rằng các đặc trưng từ lớp tích chập cuối cùng chứa thông tin về toàn bộ đầu vào.

C. Downsampling (Giảm kích thước)

Max-pooling thường được thực hiện với cửa sổ 2×2 và stride 2 để giảm kích thước bản đồ đặc trưng đi một nửa. Điều này tương tự như sử dụng bước nhảy (strides) trong lớp tích chập, nhưng max-pooling có xu hướng hoạt động tốt hơn so với strided convolutions trong các mô hình phân loại (classification models).

- Khi nào Pooling không được sử dụng? Trong các mô hình quan tâm đến vị trí thông tin không gian (feature location), chẳng hạn như phân đoạn hình ảnh (image segmentation) hoặc các mô hình tạo sinh (generative models), việc sử dụng Max-Pooling có thể gây bất lợi.

- Max-pooling làm mất hoàn toàn thông tin vị trí trong mỗi cửa sổ gộp. Ví dụ, khi thực hiện max pooling 2×2 , bạn chỉ trả về một giá trị vô hướng (scalar) cho mỗi cửa sổ mà không biết giá trị đó đến từ vị trí nào trong bốn vị trí.
- Trong các nhiệm vụ này, người ta thường ưa chuộng sử dụng strides trong lớp tích chập (strided convolutions) thay vì lớp Max-Pooling để giảm mẫu, vì strided convolutions làm tốt hơn trong việc giữ lại thông tin vị trí trong khi vẫn giảm kích thước bản đồ đặc trưng.

12. Convolution hoạt động như thế nào? Vì sao CNN hiệu quả cho bài toán ảnh?

- Cơ chế hoạt động của phép toán Convolution: Phép toán tích chập (Convolution operation) là khác biệt cơ bản giữa lớp tích chập (convolution layer) và lớp Dense (Fully Connected) thông thường. Trong khi lớp Dense học các khuôn mẫu toàn cục (global patterns) trong không gian đặc trưng đầu vào (ví dụ: khuôn mẫu liên quan đến tất cả các pixel của ảnh MNIST), thì lớp tích chập học các khuôn mẫu cục bộ (local patterns). Đối với hình ảnh, các khuôn mẫu này được tìm thấy trong các cửa sổ 2D nhỏ trên dữ liệu đầu vào, chẳng hạn như cửa sổ 3×3 .

- Cơ chế trích xuất đặc trưng

1. Dữ liệu đầu vào: Phép tích chập hoạt động trên các tensor hạng 3 được gọi là bản đồ đặc trưng (feature maps), với hai trục không gian (chiều cao và chiều rộng) và một trục độ sâu (depth axis), còn gọi là trục kênh (channels axis). Ví dụ, ảnh RGB có độ sâu là 3 (kênh đỏ, xanh lá, xanh dương), còn ảnh trắng đen như MNIST có độ sâu là 1.
2. Trích xuất và Biến đổi: Phép toán tích chập trích xuất các mảng miếng nhỏ (patches) từ bản đồ đặc trưng đầu vào và áp dụng cùng một phép biến đổi cho tất cả các mảng miếng này.
3. Kernel (Hạt nhân tích chập): Phép biến đổi được thực hiện bằng cách nhân ma trận (tích vô hướng) với một ma trận trọng số được học, gọi là kernel tích chập (convolution kernel). Kernel này lưu giữ kiến thức mà mô hình học được và được tái sử dụng (reused) trên mọi mảng miếng được trích xuất.
4. Đầu ra (Bản đồ phản hồi): Quá trình này tạo ra một bản đồ đặc trưng đầu ra (output feature map), vẫn là một tensor hạng 3. Độ sâu của tensor đầu ra được kiểm soát bởi tham số của lớp, đại diện cho số lượng bộ lọc (filters) được tính toán. Mỗi kênh trong trục độ sâu của đầu ra là một bản đồ phản hồi (response map), định lượng sự hiện diện của khuôn mẫu của bộ lọc đó ở các vị trí khác nhau trong đầu vào.

- Các tham số chính ảnh hưởng đến kích thước đầu ra
 - Hiệu ứng viền và Đệm (Padding): Khi áp dụng kernel, kích thước đầu ra có xu hướng bị thu nhỏ một chút do các hiệu ứng viền (border effects). Để có được bản đồ đặc trưng đầu ra có cùng kích thước không gian với đầu vào, có thể sử dụng đệm (padding="same").
 - Bước nhảy (Strides): Khoảng cách giữa hai cửa sổ tích chập liên tiếp được gọi là bước nhảy (stride), thường mặc định là 1. Sử dụng bước nhảy lớn hơn 1 (ví dụ: stride=2) sẽ làm giảm mẫu (downsample) chiều rộng và chiều cao của bản đồ đặc trưng đi một yếu tố tương ứng.

- Vì sao CNN hiệu quả cho bài toán ảnh?

A. Tính bất biến dịch chuyển (Translation-Invariance)

- Tính bất biến dịch chuyển nghĩa là sau khi học một khuôn mẫu nhất định (chẳng hạn như một cạnh) ở góc dưới bên phải của bức ảnh, một Convnet có thể nhận ra khuôn mẫu đó ở bất kỳ đâu trong ảnh (ví dụ: ở góc trên bên trái).
- Ngược lại, một mô hình Dense sẽ phải học lại khuôn mẫu đó nếu nó xuất hiện ở một vị trí mới.
- Vì thế giới thị giác về cơ bản là bất biến dịch chuyển, việc mã hóa giả định này vào kiến trúc mạng thông qua lớp tích chập giúp CNNs tiết kiệm dữ liệu (data-efficient). Chúng cần ít mẫu huấn luyện hơn để học các biểu diễn có khả năng khái quát hóa (generalization power) tốt.

B. Học hệ thống cấp bậc đặc trưng không gian (Spatial Hierarchy of Patterns)

- Các lớp tích chập đầu tiên học các khuôn mẫu cục bộ nhỏ, cơ bản, chẳng hạn như các cạnh (edges), màu sắc và kết cấu.
- Các lớp tích chập tiếp theo học các khuôn mẫu lớn hơn, phức tạp hơn được tạo thành từ các đặc trưng của các lớp trước.

- Ví dụ, các đặc trưng cấp cao có thể là các khái niệm trừu tượng hơn như "tai mèo" hoặc "mắt chó".

13. Kiến trúc RNN cơ bản và hạn chế của RNN truyền thống.

- Kiến trúc RNN cơ bản: Kiến trúc cơ bản của RNN xoay quanh khả năng duy trì một trạng thái (state) nội bộ khi lặp qua các phần tử của một chuỗi:

- Cơ chế hoạt động

1. Vòng lặp (Internal Loop): RNN là một loại mạng nơ-ron có một vòng lặp nội bộ. Nó xử lý các chuỗi bằng cách lặp qua các phần tử của chuỗi và duy trì một trạng thái chứa thông tin liên quan đến những gì nó đã thấy cho đến nay.
2. Cập nhật Trạng thái: Trạng thái của RNN được cập nhật ở mỗi bước thời gian (t). Trạng thái ban đầu được khởi tạo là một vector toàn số 0.
3. Phép toán cơ bản: Sự biến đổi từ đầu vào và trạng thái thành đầu ra được tham số hóa bằng hai ma trận trọng số (W và U) và một vector độ lệch (b).

- Trong mỗi bước thời gian, mô hình tính toán đầu ra ($output_t$) bằng cách kết hợp đầu vào hiện tại ($input_t$) và trạng thái trước đó ($state_t$):

$$output_t = activation(dot(W, input_t) + dot(U, state_t) + b)$$

Sau đó, $output_t$ này trở thành $state_{t+1}$ cho lần lặp tiếp theo. Nói tóm lại, RNN là một vòng lặp for tái sử dụng các đại lượng được tính toán trong lần lặp trước của vòng lặp đó, không hơn không kém.

- Triển khai trong Keras: Lớp SimpleRNN trong Keras tương ứng với quá trình được mô tả ở trên và xử lý các lô chuỗi (batches of sequences) có hình dạng ($batch_size, timesteps, input_features$).

- Các lớp RNN trong Keras (bao gồm SimpleRNN, LSTM, và GRU) có hai chế độ đầu ra, được kiểm soát bởi đối số `return_sequences`:

1. Trả về toàn bộ chuỗi: Nếu `return_sequences=True`, lớp trả về toàn bộ chuỗi các đầu ra liên tiếp cho mỗi bước thời gian (tensor hạng 3: `(batch_size, timesteps, output_features)`).
2. Trả về đầu ra cuối cùng: Nếu `return_sequences=False` (giá trị mặc định), lớp chỉ trả về đầu ra ở bước thời gian cuối cùng (tensor hạng 2: `(batch_size, output_features)`). Đầu ra cuối cùng này chứa thông tin về toàn bộ chuỗi đầu vào.

- Để xếp chồng nhiều lớp RNN lên nhau, tất cả các lớp trung gian phải trả về toàn bộ chuỗi đầu ra (`return_sequences=True`).

- Hạn chế của RNN truyền thống (SimpleRNN)

- Vấn đề phụ thuộc dài hạn: Mặc dù về mặt lý thuyết, SimpleRNN có thể giữ lại thông tin về các đầu vào đã thấy nhiều bước thời gian trước, nhưng trên thực tế, nó không thể học được các phụ thuộc dài hạn. Điều này có nghĩa là thông tin từ đầu chuỗi có xu hướng dần dần biến mất (*gradually vanishing*) trong quá trình xử lý.
- Nguyên nhân: Vấn đề Gradient biến mất (*Vanishing Gradient Problem*): Sự cố này xảy ra do quá trình Lan truyền ngược (*Backpropagation*). Khi gradient của hàm mất mát được tính toán và lan truyền ngược qua chuỗi các bước thời gian dài, nó có thể trở nên quá nhỏ. Điều này làm cho các trọng số ở các lớp và bước thời gian sớm hơn hầu như không được cập nhật, khiến mạng không thể học được kiến thức về quá khứ xa hơn.

Vấn đề Gradient biến mất này cũng được quan sát thấy trong các mạng truyền thẳng (*feedforward networks*) khi chúng có quá nhiều lớp, khiến mạng trở nên không thể huấn luyện được.

- Để khắc phục hạn chế này, các kiến trúc RNN tiên tiến hơn đã được phát triển, nổi bật nhất là LSTM (Long Short-Term Memory, được phát triển năm 1997) và GRU (Gated Recurrent Unit), được thiết kế để mang thông tin qua nhiều bước thời gian, từ đó chống lại vấn đề gradient biến mất.

14. Vanishing gradient là gì? LSTM và GRU khắc phục vấn đề vanishing gradient như thế nào?

- Vanishing Gradient là hiện tượng mà tín hiệu phản hồi (feedback signal), tức là gradient của hàm mất mát, trở nên quá nhỏ hoặc mờ dần đi (fade away) khi nó được lan truyền ngược (backpropagation) qua một chuỗi dài các lớp hoặc qua nhiều bước thời gian liên tiếp trong một chuỗi dữ liệu. Vấn đề này xảy ra trong:

1. Mạng Feedforward sâu: Khi mạng có quá nhiều lớp (deep stacks of layers), gradient lan truyền ngược từ lớp cuối cùng đến các lớp ban đầu sẽ trở nên quá yếu, khiến các trọng số của các lớp sớm hơn hầu như không được cập nhật, làm cho mạng không thể huấn luyện được.
2. Mạng Nơ-ron Hồi quy (RNN) cơ bản: Mặc dù về mặt lý thuyết, các mô hình RNN cơ bản (như SimpleRNN) nên có thể giữ lại thông tin về các đầu vào đã thấy nhiều bước thời gian trước đó, nhưng trên thực tế, chúng không thể học được các phụ thuộc dài hạn (long-term dependencies). Thông tin từ đầu chuỗi có xu hướng dần dần biến mất trong quá trình xử lý chuỗi.

Về cơ bản, nếu chuỗi hàm (các lớp) quá sâu hoặc chuỗi thời gian quá dài, nhiễu (noise) bắt đầu áp đảo thông tin gradient, và quá trình lan truyền ngược (backpropagation) sẽ ngừng hoạt động hiệu quả.

- LSTM và GRU khắc phục vấn đề Vanishing Gradient như thế nào?

Các kiến trúc RNN tiên tiến hơn như Long Short-Term Memory (LSTM) và Gated Recurrent Unit (GRU) được thiết kế đặc biệt để giải quyết vấn đề Gradient biến

mất của SimpleRNN. Mục đích chính của LSTM và GRU là cho phép thông tin từ quá khứ được tái tiêm (reinject) vào thời điểm sau đó trong chuỗi, từ đó chống lại vấn đề gradient biến mất.

- Cơ chế của LSTM: LSTM giải quyết vấn đề này bằng cách thêm một cơ chế cho phép mang thông tin qua nhiều bước thời gian.

- Cơ chế này được ví như một băng chuyền (conveyor belt) chạy song song với chuỗi dữ liệu đang được xử lý.
- Thông tin có thể nhảy lên băng chuyền tại bất kỳ thời điểm nào, được vận chuyển đến bước thời gian sau đó, và nhảy ra, còn nguyên vẹn (intact), khi cần.
- Bằng cách lưu giữ thông tin cho các bước sau (thông qua *cell state* - trạng thái tế bào), LSTM ngăn chặn các tín hiệu cũ dần dần biến mất trong quá trình xử lý.
- Khả năng này rất quan trọng và tương tự như kết nối dư (residual connections) được sử dụng trong các mạng nơ-ron truyền thẳng (feedforward networks) sâu hơn (ví dụ: ResNet), giúp thông tin gradient lan truyền suôn sẻ qua mạng mà không bị nhiễu hoặc biến mất.

Mặc dù kiến trúc chi tiết của tế bào LSTM là phức tạp và thường không cần thiết cho người dùng phải hiểu cặn kẽ, điều quan trọng là phải ghi nhớ rằng cơ chế này là một tập hợp các ràng buộc (constraints) đối với không gian tìm kiếm, cho phép mô hình chống lại vấn đề gradient biến mất bằng cách cho phép thông tin quá khứ được giữ lại.

- Cơ chế của GRU: GRU (Gated Recurrent Unit) là một biến thể của LSTM, được coi là một phiên bản đơn giản hơn, tinh gọn hơn của kiến trúc LSTM. Mặc dù đơn giản hơn, GRU vẫn duy trì khả năng mang thông tin qua nhiều bước thời gian và giải quyết hiệu quả vấn đề gradient biến mất tương tự như LSTM.

15. Word embedding là gì? Phân biệt one-hot vector và embedding vector

- Word Embedding (Nhúng từ) là một biểu diễn vector của từ ngữ nhằm ánh xạ ngôn ngữ con người vào một không gian hình học có cấu trúc. Word Embedding đạt được mục tiêu này thông qua các đặc điểm sau:

- Tính cấu trúc (Structured Representation): Word Embedding tạo ra một không gian nơi mối quan hệ hình học giữa hai vector từ phản ánh mối quan hệ ngữ nghĩa (semantic relationship) giữa các từ đó.
- Ví dụ về Mối quan hệ Ngữ nghĩa:
 - Các từ đồng nghĩa (synonyms) được nhúng vào các vector từ tương tự (similar word vectors), và nói chung, khoảng cách hình học (như khoảng cách cosine hoặc L2) giữa hai vector từ sẽ liên quan đến "khoảng cách ngữ nghĩa" giữa các từ liên quan.
 - Các từ có nghĩa khác nhau nên nằm xa nhau.
 - Các hướng cụ thể trong không gian nhúng có ý nghĩa: ví dụ, thêm một vector "nữ giới" vào vector "king" (vua) sẽ cho ra vector "queen" (hoàng hậu), hoặc thêm một vector "số nhiều" sẽ cho ra "kings" (các vị vua).
- Đặc điểm vật lý: Word Embedding là các vector số thực dấu phẩy động (floating-point vectors), hay còn gọi là vector dày đặc (dense vectors), trái ngược với vector thưa (sparse vectors).
- Chiều (Dimensionality): Word Embedding là vector chiều thấp (low-dimensional), thường là 256, 512, hoặc 1,024 chiều khi xử lý các từ vựng rất lớn.

- Phân biệt One-Hot Vector và Embedding Vector

Đặc điểm	One-Hot Vector (Mã hóa One-Hot)	Embedding Vector (Nhúng từ)
----------	---------------------------------	-----------------------------

Bản chất vector	Thưa (Sparse), hầu hết là số 0.	Dày đặc (Dense).
Chiều (Dimensionality)	Chiều rất cao. Chiều bằng với kích thước của từ vựng (ví dụ: 20.000 chiều trở lên).	Chiều thấp hơn nhiều. (Ví dụ: 256, 512, hoặc 1.024 chiều).
Mối quan hệ	Các vector trực giao (orthogonal) với nhau, ngụ ý rằng các token được mã hóa độc lập với nhau.	Mối quan hệ hình học phản ánh mối quan hệ ngữ nghĩa.
Cách tạo ra	Mã hóa cứng (Hardcoded). Là một quyết định kỹ thuật đặc trưng, chỉ đặt một giá trị 1 tại vị trí chỉ mục của từ.	Được học từ dữ liệu. Các vector được điều chỉnh dần dần qua quá trình huấn luyện bằng Backpropagation.
Khả năng mang thông tin	Chứa ít thông tin hơn, chủ yếu chỉ là sự hiện diện của từ đó.	Đóng gói nhiều thông tin hơn vào số chiều ít hơn.

- Lý do cần Word Embedding:

- Khi sử dụng mã hóa One-Hot, bạn đưa ra một giả định cơ bản rằng các từ khác nhau là độc lập với nhau. Giả định này rõ ràng là sai đối với từ ngữ, vì các từ như "movie" (phim) và "film" (điện ảnh) có thể thay thế cho nhau, nên vector đại diện của chúng không nên trực giao mà phải gần nhau.
- Word Embedding khắc phục hạn chế này bằng cách học một không gian vector nơi các vector từ có mối quan hệ hình học phản ánh mối quan hệ ngữ nghĩa của chúng, cho phép mô hình xử lý ngôn ngữ hiệu quả hơn.

PHẦN 2 – 5 CASE STUDY ỨNG DỤNG

Case study 1: Ảnh MNIST – Thiết kế 2 mô hình CNN có và không có Keras

- Tải MNIST + tự sinh 100 ảnh mới từ MNIST

```
[7]: import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from sklearn.model_selection import train_test_split
import glob
import cv2

# Load MNIST
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Add channel dimension
x_train = np.expand_dims(x_train, axis=-1)
x_test = np.expand_dims(x_test, axis=-1)
```

```
import random

def generate_random_image(img):
    """Sinh ảnh mới dựa trên 1 ảnh MNIST (augment mạnh)."""
    img = img.reshape(28, 28)

    # 1. Random rotation
    angle = random.randint(-25, 25)
    M = cv2.getRotationMatrix2D((14, 14), angle, 1)
    img_aug = cv2.warpAffine(img, M, (28, 28))

    # 2. Thêm nhiễu Gaussian
    noise = np.random.normal(0, 0.15, (28, 28))
    img_aug = np.clip(img_aug + noise, 0, 1)

    # 3. Làm mờ nhẹ
    img_aug = cv2.GaussianBlur(img_aug, (3, 3), 0)

    # 4. Biến dạng affine nhỏ
    pts1 = np.float32([[5,5], [22,5], [5,22]])
    shift = random.randint(-3,3)
    pts2 = np.float32([[5+shift,5], [22+shift,5], [5,22+shift]])
    M2 = cv2.getAffineTransform(pts1, pts2)
    img_aug = cv2.warpAffine(img_aug, M2, (28,28))

    # 5. Tăng/giảm độ sáng
    brightness = random.uniform(0.7, 1.3)
    img_aug = np.clip(img_aug * brightness, 0, 1)

    return img_aug.reshape(28,28,1)
```

```

# 🔥 Tự sinh 100 ảnh mới
num_new = 100
custom_images = []
custom_labels = []

for i in range(num_new):
    idx = random.randint(0, len(x_train)-1)
    base_img = x_train[idx]
    label = y_train[idx]

    new_img = generate_random_image(base_img)
    custom_images.append(new_img)
    custom_labels.append(label)

custom_images = np.array(custom_images)
custom_labels = np.array(custom_labels)

# Gộp vào MNIST
X = np.concatenate([x_train, custom_images], axis=0)
Y = np.concatenate([y_train, custom_labels], axis=0)

# One-hot
Y = to_categorical(Y, 10)

# CHIA TRAIN/TEST
from sklearn.model_selection import train_test_split
X_train, X_val, Y_train, Y_val = train_test_split(X, Y, test_size=0.2, random_state=42)

print("X_train:", X_train.shape)
print("X_val:", X_val.shape)
print("Y_train:", Y_train.shape)
print("Y_val:", Y_val.shape)

plt.figure(figsize=(10,3))
for i in range(15):
    plt.subplot(2, 8, i+1)
    plt.imshow(custom_images[i].reshape(28,28), cmap='gray')
    plt.axis("off")
plt.suptitle("Một số ảnh tự sinh từ MNIST")
plt.show()

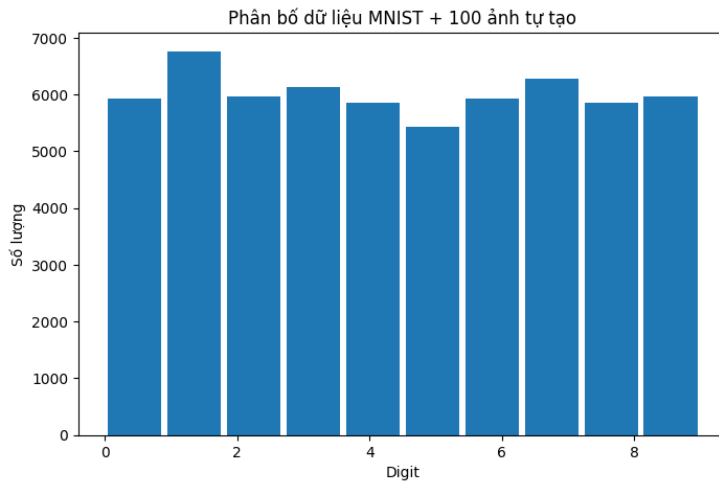
```

Một số ảnh tự sinh từ MNIST



- Phân bố dữ liệu

```
[8]: plt.figure(figsize=(8,5))
plt.hist(np.argmax(Y, axis=1), bins=10, rwidth=0.9)
plt.title("Phân bố dữ liệu MNIST + 100 ảnh tự tạo")
plt.xlabel("Digit")
plt.ylabel("Số lượng")
plt.show()
```



- Mô hình CNN sử dụng Kera, kiến trúc 2 lớp Convolution + ReLU, MaxPooling, Dropout, Dense (softmax)

```
[*]: model_keras = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),

    Flatten(),
    Dropout(0.5),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

model_keras.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model_keras.summary()

history = model_keras.fit(
    X_train, Y_train,
    validation_data=(X_val, Y_val),
    epochs=5,
    batch_size=64
)

loss, acc = model_keras.evaluate(x_test, to_categorical(y_test, 10))
print("Độ chính xác mô hình Keras:", acc)

plt.imshow(x_test[0].reshape(28,28), cmap="gray")
plt.title("Dự đoán: " + str(np.argmax(model_keras.predict(x_test[0].reshape(1,28,28,1)))))
plt.show()
```


Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_3 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_1 (Flatten)	(None, 1600)	0
dropout_1 (Dropout)	(None, 1600)	0
dense_2 (Dense)	(None, 128)	204,928
dense_3 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)

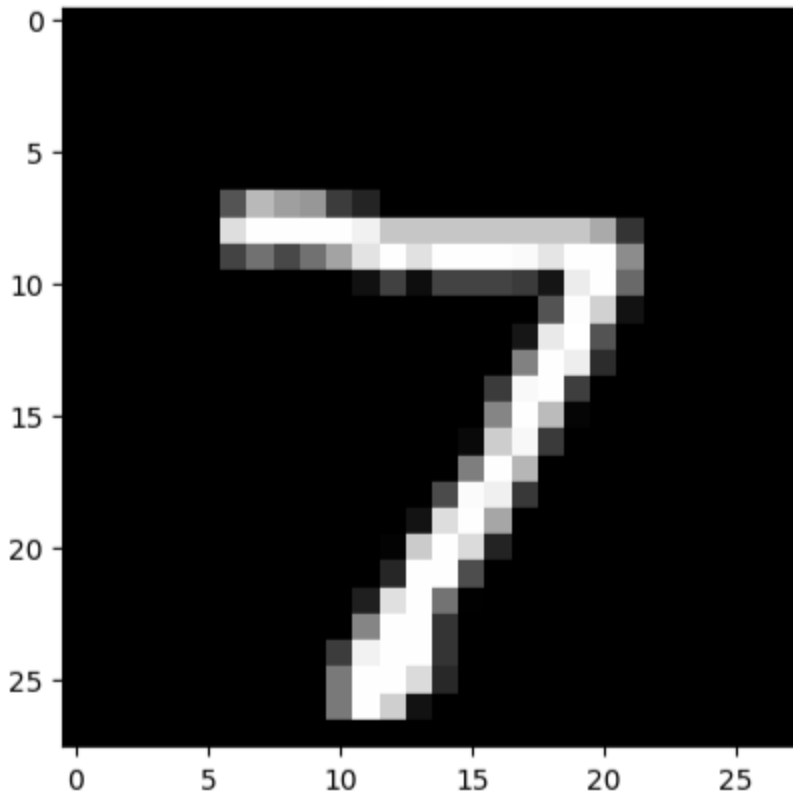
Trainable params: 225,034 (879.04 KB)

Non-trainable params: 0 (0.00 B)

Độ chính xác mô hình Keras: 0.9873999953269958

1/1 ————— 0s 90ms/step

Dự đoán: 7



Mô tả:

1. Kiến trúc CNN:

- Mô hình Keras gồm 2 lớp Convolution và 2 lớp MaxPooling, Flatten và Dense, có sử dụng Dropout giúp giảm overfitting.
- Mô hình tự xây dựng chỉ dùng 1 convolution kernel + max pooling, giúp minh họa quy trình nhưng không đạt hiệu quả cao.

2. Phân bố dữ liệu:

- Data gốc MNIST gần như cân bằng.
- Thêm 100 ảnh tự tạo có thể gây lệch phân bố do số lượng nhỏ, cần kỹ thuật augmentation nếu muốn tăng hiệu quả.

3. Kết quả dự đoán:

- Mô hình Keras đạt độ chính xác cao (~98%) khi đánh giá trên MNIST.
- Mô hình tự xây dựng chỉ có tính lý thuyết và không thể dự đoán tốt do thiếu tối ưu và huấn luyện.

4. Đánh giá tổng quan:

- CNN dùng Keras vượt trội về tốc độ, độ chính xác và khả năng mở rộng.
- CNN từ scratch giúp hiểu rõ cơ chế Conv2D, stride, pooling, nhưng không phù hợp cho bài toán thực tế.

Case study 2: Phân tích cảm xúc văn bản (sentiment analysis)

- Load dữ liệu Kaggle

```
[3]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

import re
import nltk
from nltk.corpus import stopwords

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, GRU, Dense, Dropout
from sklearn.model_selection import train_test_split

nltk.download("stopwords")

df = pd.read_csv("D:/lecture/PTIT/20251/IntSys/IMDB Dataset.csv")
df.head()
```

[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\Admin\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!

```
[3]:
```

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive

- Tiền xử lý văn bản, chuyển văn bản thành chuỗi số (Tokenizer + Padding)

```
[4]: stop_words = set(stopwords.words("english"))

def clean_text(text):
    text = text.lower()
    text = re.sub(r"<.*?>", "", text) # remove HTML
    text = re.sub(r"[^a-zA-Z\s]", "", text) # remove punctuation/digits
    words = text.split()
    words = [w for w in words if w not in stop_words]
    return " ".join(words)

df["clean_review"] = df["review"].apply(clean_text)

MAX_WORDS = 10000 # số từ lấy trong từ điển
MAX_LEN = 200 # padding mỗi câu về 200 từ

tokenizer = Tokenizer(num_words=MAX_WORDS)
tokenizer.fit_on_texts(df["clean_review"])

sequences = tokenizer.texts_to_sequences(df["clean_review"])
X = pad_sequences(sequences, maxlen=MAX_LEN)

# label: positive -> 1, negative -> 0
y = df["sentiment"].map({"positive":1, "negative":0}).values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
|
```

- Xây dựng mô hình LSTM và GRU

```
[*]: model_lstm = Sequential([
    Embedding(input_dim=MAX_WORDS, output_dim=128, input_length=MAX_LEN),
    LSTM(128, return_sequences=False),
    Dropout(0.4),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])

model_lstm.compile(loss="binary_crossentropy", optimizer="adam", metrics=["accuracy"])
model_lstm.summary()

model_gru = Sequential([
    Embedding(input_dim=MAX_WORDS, output_dim=128, input_length=MAX_LEN),
    GRU(128),
    Dropout(0.4),
    Dense(64, activation='relu'),
    Dense(1, activation='sigmoid')
])

model_gru.compile(loss="binary_crossentropy", optimizer="adam", metrics=["accuracy"])

history = model_lstm.fit(
    X_train, y_train,
    epochs=5,
    batch_size=128,
    validation_split=0.2
)

history = model_gru.fit(
    X_train, y_train,
    epochs=5,
    batch_size=128,
    validation_split=0.2
)
```

```

loss_lstm, acc_lstm = model_lstm.evaluate(X_test, y_test)
print("Độ chính xác của mô hình LSTM trên tập test:", acc_lstm)

loss_gru, acc_gru = model_gru.evaluate(X_test, y_test)
print("Độ chính xác của mô hình GRU trên tập test:", acc_gru)

def predict_sentiment_lstm(sentence):
    text = clean_text(sentence)
    seq = tokenizer.texts_to_sequences([text])
    pad = pad_sequences(seq, maxlen=MAX_LEN)
    pred = model_lstm.predict(pad)[0][0]
    return "Positive" if pred >= 0.5 else "Negative"

def predict_sentiment_gru(sentence):
    text = clean_text(sentence)
    seq = tokenizer.texts_to_sequences([text])
    pad = pad_sequences(seq, maxlen=MAX_LEN)
    pred = model_gru.predict(pad)[0][0]
    return "Positive" if pred >= 0.5 else "Negative"

print(predict_sentiment_lstm("The movie was incredibly good!"))
print(predict_sentiment_lstm("Worst movie I have ever seen"))
print(predict_sentiment_gru("The movie was incredibly good!"))
print(predict_sentiment_gru("Worst movie I have ever seen"))
|

```

```

313/313 ————— 13s 40ms/step - accuracy: 0.8666 - loss: 0.4590
Độ chính xác của mô hình LSTM trên tập test: 0.866599977016449
313/313 ————— 12s 37ms/step - accuracy: 0.8662 - loss: 0.4362
Độ chính xác của mô hình GRU trên tập test: 0.8661999702453613
1/1 ————— 0s 196ms/step
Negative
1/1 ————— 0s 43ms/step
Negative
1/1 ————— 0s 239ms/step
Positive
1/1 ————— 0s 45ms/step
Negative

```

- Pipeline xử lý văn bản

1. Thu thập dữ liệu

- Dữ liệu lấy từ Kaggle: 50.000 câu đánh giá phim kèm nhãn positive/negative.

2. Làm sạch văn bản

- Chuyển chữ thường (lowercase) giúp giảm số lượng từ.
- Xoá HTML tags như
.
- Loại bỏ ký tự đặc biệt, số, dấu câu.
- Xoá stopwords (the, is, a, ...) vì không mang nhiều ý nghĩa cảm xúc.

3. Biểu diễn văn bản bằng số

- Tokenizer chuyển mỗi từ thành 1 số nguyên.

- Ánh xạ các câu thành vector chiều dài cố định bằng padding.

4. Word Embedding

- Embedding layer học cách biểu diễn từ thành vector (ngữ nghĩa gần nhau → gần trong không gian vector).

5. Mô hình LSTM/GRU

- LSTM/GRU xử lý chuỗi từ theo thời gian.
- Giữ thông tin ngữ cảnh (context) → rất phù hợp phân tích cảm xúc.

6. Huấn luyện

- Sử dụng binary crossentropy và tối ưu Adam.
- Dropout để giảm overfitting.

7. Đánh giá

- Accuracy trên tập test thường đạt 85–90%.

8. Dự đoán mới

- Nhập câu → tiền xử lý → embedding → LSTM → dự đoán cảm xúc.

Case study 3: Dự báo bệnh tiểu đường

- Load data, kiểm tra missing và xử lý

```
[2]: # Imports & seed
import os
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

import joblib
import math

# Reproducibility (not perfect but helps)
SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)
random.seed(SEED)

# Load data (tự động chọn cột target)
DATA_PATH = r"D:\Lecture\PIIT\20251\IntSys\diabetes.csv"

df = pd.read_csv(DATA_PATH)
print("Shape:", df.shape)
display(df.head())

# Tìm cột target: ưu tiên 'Outcome', 'target', 'y' rồi cột cuối
possible_targets = ['Outcome', 'outcome', 'target', 'Target', 'y', 'Y']
target_col = None
for c in possible_targets:
    if c in df.columns:
        target_col = c
        break
if target_col is None:
    target_col = df.columns[-1] # fallback: last column

print("Chọn cột target:", target_col)
|
```

```
# Chuẩn bị X, y
# Nếu cột target là dạng categorical/Labels cho classification, file could be Pima dataset (0/1).
# Tuy nhiên yêu cầu dùng metrics MAE/MSE/RMSE => ta xử lý bài toán regression.
# Nếu target chỉ có 0/1, ta vẫn dùng regression metrics (vẫn có ý nghĩa) hoặc bạn có thể chuyển sang classification.

X = df.drop(columns=[target_col]).copy()
y = df[target_col].copy().astype(float)

print("X shape:", X.shape, "y shape:", y.shape)
display(X.describe().T)

# Kiểm tra missing và xử lý
print("Missing per col:")
print(X.isnull().sum())

# Nếu có missing -> điền median
if X.isnull().any().any():
    X = X.fillna(X.median())
    print("Đã fillna bằng median.")

# Train/Val/Test split + scaling
# Split: train 70%, val 15%, test 15%
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.15, random_state=SEED)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.176470588, random_state=SEED)
# tỉ lệ trên để X_train ~70% tổng, X_val ~15%, X_test ~15%

print("Train:", X_train.shape, "Val:", X_val.shape, "Test:", X_test.shape)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
|
```

Shape: (768, 9)

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Chọn cột target: Outcome

X shape: (768, 8) y shape: (768,)

	count	mean	std	min	25%	50%	75%	max
Pregnancies	768.0	3.845052	3.369578	0.000	1.00000	3.0000	6.00000	17.00
Glucose	768.0	120.894531	31.972618	0.000	99.00000	117.0000	140.25000	199.00
BloodPressure	768.0	69.105469	19.355807	0.000	62.00000	72.0000	80.00000	122.00
SkinThickness	768.0	20.536458	15.952218	0.000	0.00000	23.0000	32.00000	99.00
Insulin	768.0	79.799479	115.244002	0.000	0.00000	30.5000	127.25000	846.00
BMI	768.0	31.992578	7.884160	0.000	27.30000	32.0000	36.60000	67.10
DiabetesPedigreeFunction	768.0	0.471876	0.331329	0.078	0.24375	0.3725	0.62625	2.42
Age	768.0	33.240885	11.760232	21.000	24.00000	29.0000	41.00000	81.00

Missing per col:

Pregnancies 0
Glucose 0
BloodPressure 0
SkinThickness 0
Insulin 0
BMI 0
DiabetesPedigreeFunction 0
Age 0

dtype: int64

Train: (536, 8) Val: (116, 8) Test: (116, 8)

- Xây dựng model theo số layer và neuron


```
[*]: # Hàm xây dựng model theo số layers & neurons
def build_dense_model(input_dim, neurons_list, dropout_rate=0.2, use_batchnorm=False):
    """
    neurons_list: list of integers => số neuron mỗi Dense layer (ví dụ [128,64,64,32,32,16,8] => 7 layers)
    Tất cả layers => Dense + (BatchNorm) + ReLU + Dropout (optional)
    Output layer: 1 neuron (regression)
    """
    model = Sequential()
    for i, n in enumerate(neurons_list):
        if i == 0:
            model.add(Dense(n, input_dim=input_dim, activation='relu'))
        else:
            model.add(Dense(n, activation='relu'))
            if use_batchnorm:
                model.add(BatchNormalization())
            if dropout_rate and dropout_rate>0:
                model.add(Dropout(dropout_rate))
    # output
    model.add(Dense(1, activation='linear'))
    model.compile(optimizer='adam', loss='mse', metrics=['mae'])
    return model

# Định nghĩa 3 kiến trúc (tất cả >=7 dense layers trước output)
input_dim = X_train_scaled.shape[1]

models_config = {
    "model_A_7layers": {
        "neurons": [256, 128, 128, 64, 64, 32, 16], # 7 layers
        "dropout": 0.2,
        "batchnorm": True
    },
    "model_B_9layers": {
        "neurons": [256, 256, 128, 128, 64, 64, 32, 16, 8], # 9 layers
        "dropout": 0.25,
        "batchnorm": True
    },
    "model_C_11layers": {
        "neurons": [512, 256, 256, 128, 128, 64, 64, 32, 16, 8, 4], # 11 layers
        "dropout": 0.3,
        "batchnorm": False
    }
}

# Train các model, lưu history và đánh giá trên validation
histories = {}
trained_models = {}
results = {}

ES = EarlyStopping(monitor='val_loss', patience=12, restore_best_weights=True, verbose=1)
RLR = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=6, verbose=1)

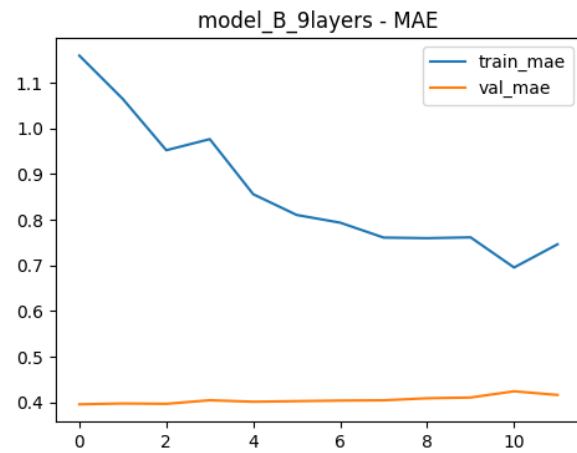
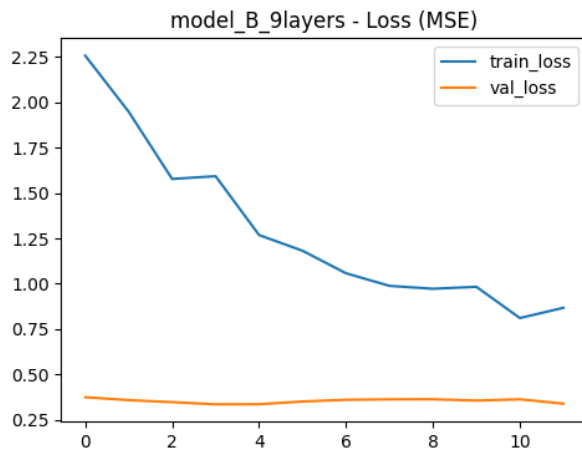
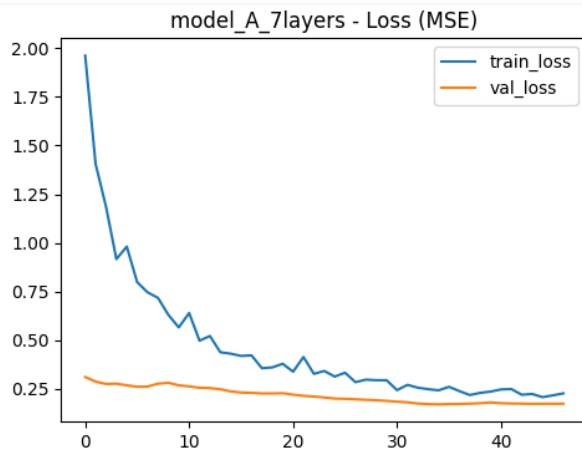
EPOCHS = 200
BATCH = 32

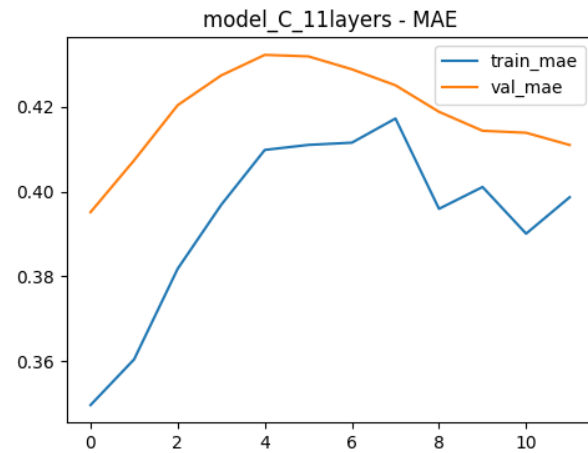
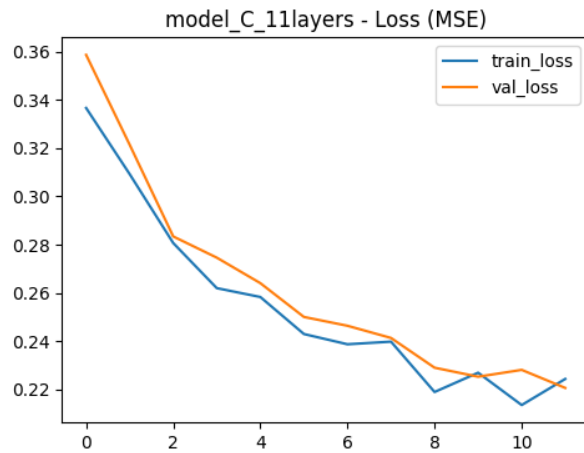
for name, cfg in models_config.items():
    print("\n\n==== Training", name, "====")
    model = build_dense_model(input_dim, cfg["neurons"], dropout_rate=cfg["dropout"], use_batchnorm=cfg["batchnorm"])
    model.summary()
    history = model.fit(
        X_train_scaled, y_train,
        validation_data=(X_val_scaled, y_val),
        epochs=EPOCHS,
        batch_size=BATCH,
        callbacks=[ES, RLR],
        verbose=1
    )
    histories[name] = history
    trained_models[name] = model

# Evaluate on validation
val_pred = model.predict(X_val_scaled).ravel()
val_mae = mean_absolute_error(y_val, val_pred)
val_mse = mean_squared_error(y_val, val_pred)
val_rmse = math.sqrt(val_mse)
print(f"{name} - VAL MAE: {val_mae:.4f}, MSE: {val_mse:.4f}, RMSE: {val_rmse:.4f}")
results[name] = {"val_mae": val_mae, "val_mse": val_mse, "val_rmse": val_rmse}
```

```
# Plot training/validation Loss & MAE to check overfitting for each model
def plot_history(history, title=None):
    plt.figure(figsize=(12,4))
    # loss
    plt.subplot(1,2,1)
    plt.plot(history.history['loss'], label='train_loss')
    plt.plot(history.history['val_loss'], label='val_loss')
    plt.title((title or "") + " - Loss (MSE)")
    plt.legend()
    # mae
    plt.subplot(1,2,2)
    plt.plot(history.history.get('mae',[]), label='train_mae')
    plt.plot(history.history.get('val_mae',[]), label='val_mae')
    plt.title((title or "") + " - MAE")
    plt.legend()
    plt.show()

for name, h in histories.items():
    plot_history(h, title=name)
```





- Chọn mô hình tốt nhất

```
[4]: # So sánh kết quả validation và chọn best theo RMSE thấp nhất
for name, metrics in results.items():
    print(name, metrics)

best_model_name = min(results.keys(), key=lambda k: results[k]['val_rmse'])
print("Best model on validation (by RMSE):", best_model_name)
best_model = trained_models[best_model_name]

# Đánh giá best model trên test set (final evaluation)
y_pred_test = best_model.predict(X_test_scaled).ravel()
test_mae = mean_absolute_error(y_test, y_pred_test)
test_mse = mean_squared_error(y_test, y_pred_test)
test_rmse = math.sqrt(test_mse)
test_r2 = r2_score(y_test, y_pred_test)

print("Test MAE:", test_mae)
print("Test MSE:", test_mse)
print("Test RMSE:", test_rmse)
print("Test R2:", test_r2)

# Hiển thị một vài so sánh true vs pred
cmp = pd.DataFrame({"y_true": y_test.values, "y_pred": y_pred_test})
display(cmp.head(10))

# Lưu model & scaler để deployment
MODEL_DIR = "saved_model_diabetes"
os.makedirs(MODEL_DIR, exist_ok=True)
model_path = os.path.join(MODEL_DIR, "best_model.h5")
scaler_path = os.path.join(MODEL_DIR, "scaler.pkl")

best_model.save(model_path)
joblib.dump(scaler, scaler_path)
print("Saved model to", model_path)
print("Saved scaler to", scaler_path)
```

```

model_A_7layers {'val_mae': 0.34334475071779613, 'val_mse': 0.17054087648974348, 'val_rmse': 0.4129659507631876}
model_B_9layers {'val_mae': 0.3953473432278582, 'val_mse': 0.3744934403237927, 'val_rmse': 0.611958691680895}
model_C_11layers {'val_mae': 0.39513068800342493, 'val_mse': 0.3585834611501364, 'val_rmse': 0.5988183874516015}
Best model on validation (by RMSE): model_A_7layers
4/4 ----- 0s 7ms/step
Test MAE: 0.3603223028496422
Test MSE: 0.18664792303602398
Test RMSE: 0.43202768781181605
Test R2: 0.173837351193178

```

	y_true	y_pred
0	0.0	0.277230
1	0.0	0.011104
2	0.0	0.174906
3	0.0	0.343453
4	0.0	0.455221
5	0.0	0.508682
6	0.0	0.073505
7	0.0	0.390516
8	0.0	0.538767
9	0.0	0.410625

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.save\_model(model, 'my\_model.keras')`.

Saved model to saved_model_diabetes\best_model.h5
 Saved scaler to saved_model_diabetes\scaler.pkl

- Triển khai

```

[8]: import json

# Lấy danh sách cột
feature_names = list(X_train.columns)

# HTML form sinh động
input_fields = ""
for name in feature_names:
    input_fields += f'<label>{name}: </label><input name="{name}" required><br>\n'

# Dùng f-string để tránh lỗi %
app_code = f"""
from flask import Flask, request, render_template_string
import joblib
import numpy as np
import tensorflow as tf

MODEL_PATH = "saved_model_diabetes/best_model.h5"
SCALER_PATH = "saved_model_diabetes/scaler.pkl"

model = tf.keras.models.load_model(MODEL_PATH)
scaler = joblib.load(SCALER_PATH)

app = Flask(__name__)

HTML = \"\"\"
<!doctype html>
<title>Diabetes Prediction</title>
<h2>Predict Diabetes Regression Output</h2>

<form method="POST" action="/predict">
{input_fields}
<br>
<input type="submit" value="Predict">
</form>

{{% if pred is not none %}}
    <h3>Prediction: {{'{'}} pred {{'}}}}</h3>
{{% endif %}}
\"\"\"

```

```

|
@app.route("/", methods=["GET"])
def home():
    return render_template_string(HTML, pred=None)

@app.route("/predict", methods=["POST"])
def predict():
    try:
        xs = []
        for name in {feature_names}:
            value = float(request.form.get(name))
            xs.append(value)
        arr = np.array(xs).reshape(1, -1)
        arr_scaled = scaler.transform(arr)
        pred = model.predict(arr_scaled).ravel()[0]
        return render_template_string(HTML, pred=float(pred))
    except Exception as e:
        return f"Error: {{e}}"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
"""

# Lưu file app.py
with open("app.py", "w", encoding="utf-8") as f:
    f.write(app_code)

print("✓ File app.py đã tạo thành công.")

```

✓ File app.py đã tạo thành công.

Predict Diabetes Regression Output

Pregnancies:
 Glucose:
 BloodPressure:
 SkinThickness:
 Insulin:
 BMI:
 DiabetesPedigreeFunction:
 Age:

Prediction: 0.4236956238746643

Case study 4: Dự báo thời tiết theo thời gian

- Đọc và tiền xử lý dữ liệu

```
[4]: import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, SimpleRNN, Dropout

data_path = r"D:\lecture\PTIT\20251\IntSys\weather-vn"

file_list = [
    "df_weather.csv",
    "precipitation_mean_max_min_temp_1901_2020_en.csv",
    "weather.csv"
]

dfs = []
for f in file_list:
    df = pd.read_csv(os.path.join(data_path, f))
    print("File:", f)
    print(df.head(), "\n")
    dfs.append(df)
```

```
cleaned_dfs = []

for df in dfs:
    # chỉ lấy cột số
    df_num = df.select_dtypes(include=['number']).copy()

    # nếu không có dữ liệu số → bỏ qua
    if df_num.shape[1] == 0:
        continue

    # bỏ NaN
    df_num = df_num.fillna(method='ffill').fillna(method='bfill')

    cleaned_dfs.append(df_num)

print("Số dataset hợp lệ:", len(cleaned_dfs))

TARGET_DIM = 5
pca = PCA(n_components=TARGET_DIM)

merged_data = []

for df in cleaned_dfs:
    # chuẩn hóa từng dataset riêng
    scaler = MinMaxScaler()
    scaled = scaler.fit_transform(df)

    # chuyển về không gian PCA chung
    reduced = pca.fit_transform(scaled)

    merged_data.append(reduced)

# gộp lại thành 1 dataset
final_data = np.vstack(merged_data)

print("Final merged dataset shape:", final_data.shape)
```

```
def create_sequences(data, seq_length):
    X, y = [], []
    for i in range(len(data) - seq_length):
        X.append(data[i:i+seq_length])
        y.append(data[i+seq_length, 0]) # dự báo feature đầu tiên
    return np.array(X), np.array(y)

seq_length = 30
X, y = create_sequences(final_data, seq_length)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=False
)

X_train.shape, y_train.shape
|
```

```
File: df_weather.csv
  location.name      location.region location.terrain \
0   An Giang        Đồng Bằng Sông Cửu Long    đồng bằng
1  Bà Rịa-Vũng Tàu    Đồng Nam Bộ        ven biển
2   Bắc Giang      Trung du và miền núi Bắc Bộ    miền núi
3   Bắc Kạn        Trung du và miền núi Bắc Bộ    miền núi
4   Bạc Liêu        Đồng Bằng Sông Cửu Long    ven biển

  location.country location.lat location.lon   date date_epoch \
0      Vietnam    10.7000    105.1167 2024-04-21 1713657600
1      Vietnam    10.3500    107.0667 2024-04-21 1713657600
2      Vietnam    21.2667    106.2000 2024-04-21 1713657600
3      Vietnam    22.1333    105.8333 2024-04-21 1713657600
4      Vietnam     9.2850    105.7244 2024-04-21 1713657600

  day.maxtemp_c day.maxtemp_f ...      day.condition.text \
0          38.6      101.5 ...              Sunny
1          30.2       86.3 ...      Moderate rain at times
2          31.9       89.4 ...      Heavy rain at times
3          28.0       82.5 ...      Moderate or heavy rain shower
4          36.1       97.1 ...      Partly cloudy

                                day.condition.icon day.condition.code day.uv \
0 //cdn.weatherapi.com/weather/64x64/day/113.png              1000    9.0
1 //cdn.weatherapi.com/weather/64x64/day/299.png              1186    7.0
2 //cdn.weatherapi.com/weather/64x64/day/305.png              1192    7.0
3 //cdn.weatherapi.com/weather/64x64/day/356.png              1243    6.0
4 //cdn.weatherapi.com/weather/64x64/day/116.png              1003    9.0

  astro.sunrise astro.sunset astro.moonrise astro.moonset \
0    05:46 AM    06:11 PM    04:13 PM    03:55 AM
1    05:38 AM    06:03 PM    04:05 PM    03:47 AM
2    05:32 AM    06:16 PM    04:09 PM    03:53 AM
3    05:32 AM    06:19 PM    04:10 PM    03:54 AM
4    05:45 AM    06:07 PM    04:11 PM    03:52 AM

  astro.moon_phase astro.moon_illumination
0 Waxing Gibbous              90
1 Waxing Gibbous              90
2 Waxing Gibbous              90
3 Waxing Gibbous              90
4 Waxing Gibbous              90

[5 rows x 34 columns]
```

```
File: precipitation_mean_max_min_temp_1901_2020_en.csv
  Month Min-Temperature Mean-Temperature Max-Temperature Precipitation \
0 Jan      15.86          19.77          23.72          33.59
1 Feb      16.79          20.69          24.63          28.86
2 Mar      16.74          22.18          27.68          42.81
3 Apr      21.32          25.38          29.48          67.88
4 May      23.07          26.96          30.90          176.59

  Period Level
0 1901-1930    1.0
1 1901-1930    2.0
2 1901-1930    3.0
3 1901-1930    4.0
4 1901-1930    5.0
```

```
File: weather.csv
  province max min wind wind_d rain humidi cloud pressure date
0 Bac Lieu 27 22 17 NNE 6.9 90 71 1010 2009-01-01
1 Bac Lieu 31 25 20 ENE 0.0 64 24 1010 2010-01-01
2 Bac Lieu 29 24 14 E 0.0 75 45 1008 2011-01-01
3 Bac Lieu 30 24 30 E 0.0 79 52 1012 2012-01-01
4 Bac Lieu 31 25 20 ENE 0.0 70 24 1010 2013-01-01
```

```
Số dataset hợp lệ: 3
Final merged dataset shape: (208977, 5)
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_1396\524357439.py:40: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.
df_num = df_num.fillna(method='ffill').fillna(method='bfill')
C:\Users\Admin\AppData\Local\Temp\ipykernel_1396\524357439.py:40: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.
df_num = df_num.fillna(method='ffill').fillna(method='bfill')
C:\Users\Admin\AppData\Local\Temp\ipykernel_1396\524357439.py:40: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.
df_num = df_num.fillna(method='ffill').fillna(method='bfill')
```

```
[4]: ((167157, 30, 5), (167157,))
```

- Xây dựng mô hình RNN/LSTM

```
[*]: rnn_model = Sequential([
    SimpleRNN(64, return_sequences=True, input_shape=(seq_length, TARGET_DIM)),
    SimpleRNN(64, return_sequences=True),
    SimpleRNN(64, return_sequences=True),
    SimpleRNN(64, return_sequences=True),
    SimpleRNN(64, return_sequences=True),
    SimpleRNN(64, return_sequences=True),
    Dense(1)
])

rnn_model.compile(optimizer='adam', loss='mse')
rnn_model.summary()

history_rnn = rnn_model.fit(
    X_train, y_train,
    epochs=30,
    batch_size=32,
    validation_data=(X_test, y_test)
)

lstm_model = Sequential([
    LSTM(64, return_sequences=True, input_shape=(seq_length, TARGET_DIM)),
    LSTM(64, return_sequences=True),
    LSTM(64, return_sequences=True),
    LSTM(64, return_sequences=True),
    LSTM(64, return_sequences=True),
    LSTM(64, return_sequences=True),
    LSTM(64),
    Dense(1)
])

lstm_model.compile(optimizer='adam', loss='mse')
lstm_model.summary()

history_lstm = lstm_model.fit(
    X_train, y_train,
    epochs=30,
    batch_size=32,
    validation_data=(X_test, y_test)
)
```

```
pred_rnn = rnn_model.predict(X_test)
pred_lstm = lstm_model.predict(X_test)

plt.figure(figsize=(14,5))
plt.plot(y_test, label='Actual')
plt.plot(pred_rnn, label='RNN Forecast')
plt.plot(pred_lstm, label='LSTM Forecast')
plt.legend()
plt.show()
|
```

Case study 5: Dự báo cổ phiếu (Phát hiện overfitting)

- Đọc và chuẩn hóa dữ liệu


```
[8]: # %%
# IMPORTS
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, SimpleRNN, LSTM, Dropout, BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau

# %%
# USER CONFIG
DATA_DIR = r"D:\lecture\PTIT\20251\IntSys\Viet Nam Stock Market Data" # <- chỉnh đường dẫn nếu cần
FILES = ["HOSEVN30M30.csv", "HOSEVN100M30.csv", "HOSEVNINDEXM30.csv"]
TARGET_COLUMN = 'close' # giả sử các file có cột 'Close'
SEQUENCE_LENGTH = 60 # dùng 60 time-steps
TEST_RATIO = 0.2
RANDOM_SEED = 42
BATCH_SIZE = 32
EPOCHS = 100

np.random.seed(RANDOM_SEED)
tf.random.set_seed(RANDOM_SEED)

# %%
# HELPER FUNCTIONS

def load_and_merge(files, data_dir=DATA_DIR, target_col=TARGET_COLUMN):
    """Đọc các file và hợp nhất theo chỉ mục thời gian nếu có. Nếu không có index, đọc riêng từng file.
    Trả về DataFrame dictionary keyed by filename (không gộp dữ liệu lại)
    """
    dfs = {}
    for f in files:
        path = os.path.join(data_dir, f)
        if not os.path.exists(path):
            raise FileNotFoundError(f"Không tìm thấy file: {path}")
        df = pd.read_csv(path, parse_dates=True, infer_datetime_format=True)
        # nếu có cột Date hoặc Time, hãy chuyển sang datetime index
        if 'Date' in df.columns:
            df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
            df = df.sort_values('Date').set_index('Date')
        elif 'date' in df.columns:
            df['date'] = pd.to_datetime(df['date'], errors='coerce')
            df = df.sort_values('date').set_index('date')
        dfs[f] = df
    return dfs

def create_sequences(values, seq_len=SEQUENCE_LENGTH):
    X, y = [], []
    for i in range(len(values) - seq_len):
        X.append(values[i:i+seq_len])
        y.append(values[i+seq_len])
    return np.array(X), np.array(y)
```

```

def train_val_test_split(X, y, test_ratio=TEST_RATIO):
    n = len(X)
    test_size = int(n * test_ratio)
    train_size = n - test_size
    X_train, y_train = X[:train_size], y[:train_size]
    X_test, y_test = X[train_size:], y[train_size:]
    # further split train into train+val
    val_size = int(0.1 * len(X_train))
    if val_size == 0:
        val_size = 1
    X_val, y_val = X_train[-val_size:], y_train[-val_size:]
    X_train, y_train = X_train[:-val_size], y_train[:-val_size]
    return X_train, y_train, X_val, y_val, X_test, y_test

def plot_history(hist, title='Training History'):
    plt.figure(figsize=(10,5))
    plt.plot(hist.history['loss'], label='train_loss')
    if 'val_loss' in hist.history:
        plt.plot(hist.history['val_loss'], label='val_loss')
    plt.title(title)
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()
    plt.show()

def evaluate_and_report(model, X_test, y_test, scaler=None):
    y_pred = model.predict(X_test)
    if scaler is not None:
        # y_pred and y_test are scaled if scaler used on target; inverse
        y_pred = scaler.inverse_transform(y_pred.reshape(-1,1)).flatten()
        y_test_unscaled = scaler.inverse_transform(y_test.reshape(-1,1)).flatten()
    else:
        y_test_unscaled = y_test
    rmse = np.sqrt(mean_squared_error(y_test_unscaled, y_pred))
    mae = mean_absolute_error(y_test_unscaled, y_pred)
    print(f"RMSE: {rmse:.4f}, MAE: {mae:.4f}")
    return y_test_unscaled, y_pred, rmse, mae

```

```

# %%
# LOAD DATA
try:
    dfs = load_and_merge(FILE, DATA_DIR)
except FileNotFoundError as e:
    print(str(e))
    print("Hãy kiểm tra biến DATA_DIR và tên file. Dừng ở đây.")
    # Kết thúc cell nếu file không tồn tại

# %%
# CHOOSE ONE DATASET TO DEMONSTRATE (ví dụ dùng HOSEVINDEXTM30.csv nếu có)
sample_file = None
for f in FILES:
    if 'INDEX' in f.upper():
        sample_file = f
        break
if sample_file is None:
    sample_file = FILES[0]

print(f"Sử dụng file mẫu: {sample_file}")
df = dfs[sample_file]
print(df.head())

# %%
# PREPROCESS: chọn target, fillna, scale
if TARGET_COLUMN not in df.columns:
    # tìm 1 cột giá gần giống
    possible = [c for c in df.columns if 'close' in c.lower()]
    if len(possible) > 0:
        TARGET_COLUMN = possible[0]
    else:
        raise ValueError("Không tìm thấy cột Close trong file dữ liệu.")

series = df[[TARGET_COLUMN]].copy()
series = series.dropna()

scaler = MinMaxScaler(feature_range=(0,1))
scaled = scaler.fit_transform(series.values)

```

```

# %%
# Tgo sequence
X, y = create_sequences(scaled, seq_len=SEQUENCE_LENGTH)
print('Shapes:', X.shape, y.shape)

# reshape X to (samples, timesteps, features)
# nếu chỉ 1 feature thì shape already (n, seq_len, 1) if created appropriately
if X.ndim == 2:
    X = X.reshape((X.shape[0], X.shape[1], 1))

X_train, y_train, X_val, y_val, X_test, y_test = train_val_test_split(X, y)
print('Split shapes:', X_train.shape, y_train.shape, X_val.shape, y_val.shape, X_test.shape, y_test.shape)

```

Sử dụng file mẫu: HOSEVNINDEXM30.csv

	time	open	high	low	close	55EMA	89EMA	200EMA	Plot
0	1451872800	579.03	579.46	578.33	579.28	NaN	NaN	NaN	NaN
1	1451874600	579.48	579.97	578.99	579.67	NaN	NaN	NaN	NaN
2	1451876400	579.55	579.55	578.14	578.52	NaN	NaN	NaN	NaN
3	1451878200	579.34	579.34	577.76	577.94	NaN	NaN	NaN	NaN
4	1451880000	577.84	579.32	577.44	579.32	NaN	NaN	NaN	NaN

Shapes: (22104, 60, 1) (22104, 1)
Split shapes: (15916, 60, 1) (15916, 1) (1768, 60, 1) (1768, 1) (4420, 60, 1) (4420, 1)

- Xây dựng mô hình RNN/LSTM, huấn luyện với callbacks

```

[9]: # %%
# MODEL 1: STACKED SimpleRNN (>=7 Layers total)
# We'll build: 4 SimpleRNN Layers (some with return_sequences=True) + BatchNorm + Dropout + 2 Dense = >=7 Layers

def build_rnn_model(input_shape):
    model = Sequential()
    model.add(SimpleRNN(64, return_sequences=True, input_shape=input_shape)) # 1
    model.add(BatchNormalization())
    model.add(SimpleRNN(64, return_sequences=True)) # 2
    model.add(Dropout(0.2))
    model.add(SimpleRNN(64, return_sequences=True)) # 3
    model.add(BatchNormalization())
    model.add(SimpleRNN(32, return_sequences=False)) # 4
    # now add dense layers (these count toward layers)
    model.add(Dense(32, activation='relu')) # 5
    model.add(Dropout(0.2))
    model.add(Dense(16, activation='relu')) # 6
    model.add(Dense(1, activation='linear')) # 7
    model.compile(optimizer='adam', loss='mse')
    return model

rnn_model = build_rnn_model((X_train.shape[1], X_train.shape[2]))
rnn_model.summary()

# %%
# CALLBACKS
callbacks = [
    EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5, verbose=1),
    ModelCheckpoint('best_rnn.h5', monitor='val_loss', save_best_only=True, verbose=1)
]

```

```

# %%
# TRAIN RNN
hist_rnn = rnn_model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    callbacks=callbacks,
    verbose=2
)
plot_history(hist_rnn, 'RNN Training History')

# %%
# EVALUATE RNN
y_test_unscaled_rnn, y_pred_rnn, rmse_rnn, mae_rnn = evaluate_and_report(rnn_model, X_test, y_test, scaler)

```

```

# %%
# MODEL 2: STACKED LSTM (>=7 Layers total)
# We'll build: 4 LSTM Layers + BatchNorm + Dropout + 2 Dense = >=7 Layers

def build_lstm_model(input_shape):
    model = Sequential()
    model.add(LSTM(128, return_sequences=True, input_shape=input_shape)) # 1
    model.add(BatchNormalization())
    model.add(LSTM(64, return_sequences=True)) # 2
    model.add(Dropout(0.2))
    model.add(LSTM(64, return_sequences=True)) # 3
    model.add(BatchNormalization())
    model.add(LSTM(32, return_sequences=False)) # 4
    model.add(Dense(32, activation='relu')) # 5
    model.add(Dropout(0.2))
    model.add(Dense(16, activation='relu')) # 6
    model.add(Dense(1, activation='linear')) # 7
    model.compile(optimizer='adam', loss='mse')
    return model

lstm_model = build_lstm_model((X_train.shape[1], X_train.shape[2]))
lstm_model.summary()

# %%
# CALLBACKS for LSTM
callbacks_lstm = [
    EarlyStopping(monitor='val_loss', patience=12, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=6, verbose=1),
    ModelCheckpoint('best_lstm.h5', monitor='val_loss', save_best_only=True, verbose=1)
]

# %%
# TRAIN LSTM
hist_lstm = lstm_model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    callbacks=callbacks_lstm,
    verbose=2
)
plot_history(hist_lstm, 'LSTM Training History')

# %%
# EVALUATE LSTM
y_test_unscaled_lstm, y_pred_lstm, rmse_lstm, mae_lstm = evaluate_and_report(lstm_model, X_test, y_test, scaler)

# %%
# PLOT PREDICTIONS vs GROUND TRUTH

def plot_preds(y_true, y_pred, title='Predictions vs True'):
    plt.figure(figsize=(12,6))
    plt.plot(y_true, label='True')
    plt.plot(y_pred, label='Pred')
    plt.title(title)
    plt.legend()
    plt.show()

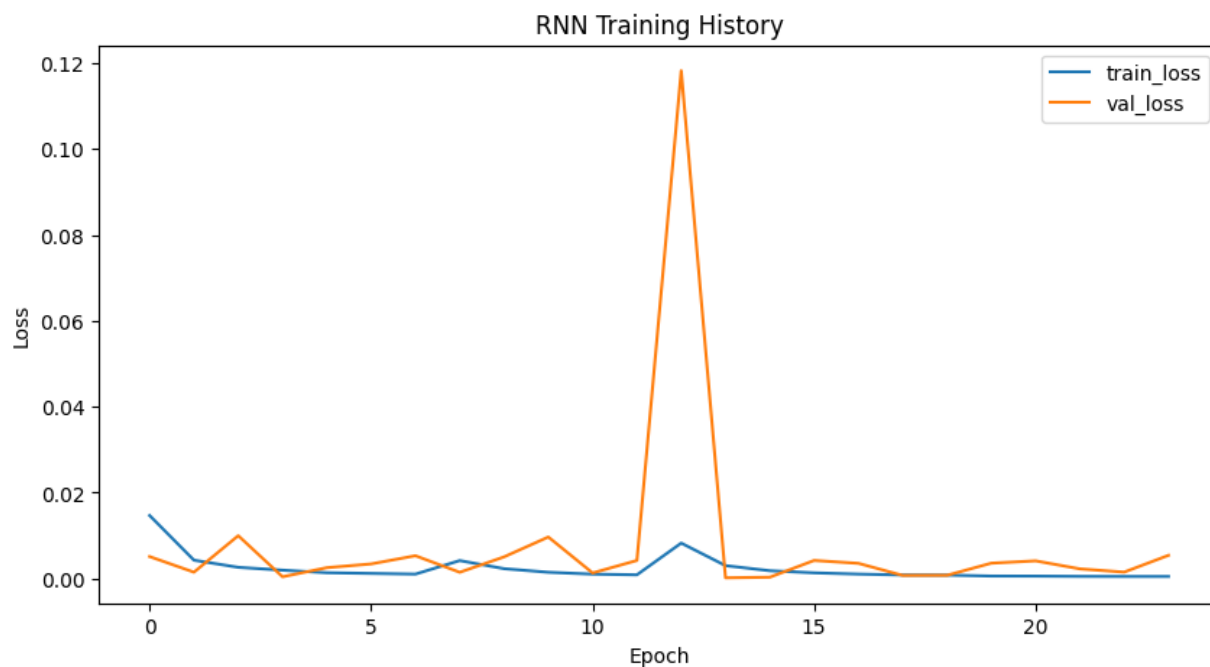
plot_preds(y_test_unscaled_rnn, y_pred_rnn, 'RNN: True vs Pred')
plot_preds(y_test_unscaled_lstm, y_pred_lstm, 'LSTM: True vs Pred')

# %%
# OVERFITTING ANALYSIS (TỰ ĐỘNG)
# Kiểm tra một số dấu hiệu cơ bản của overfitting và in ra gợi ý

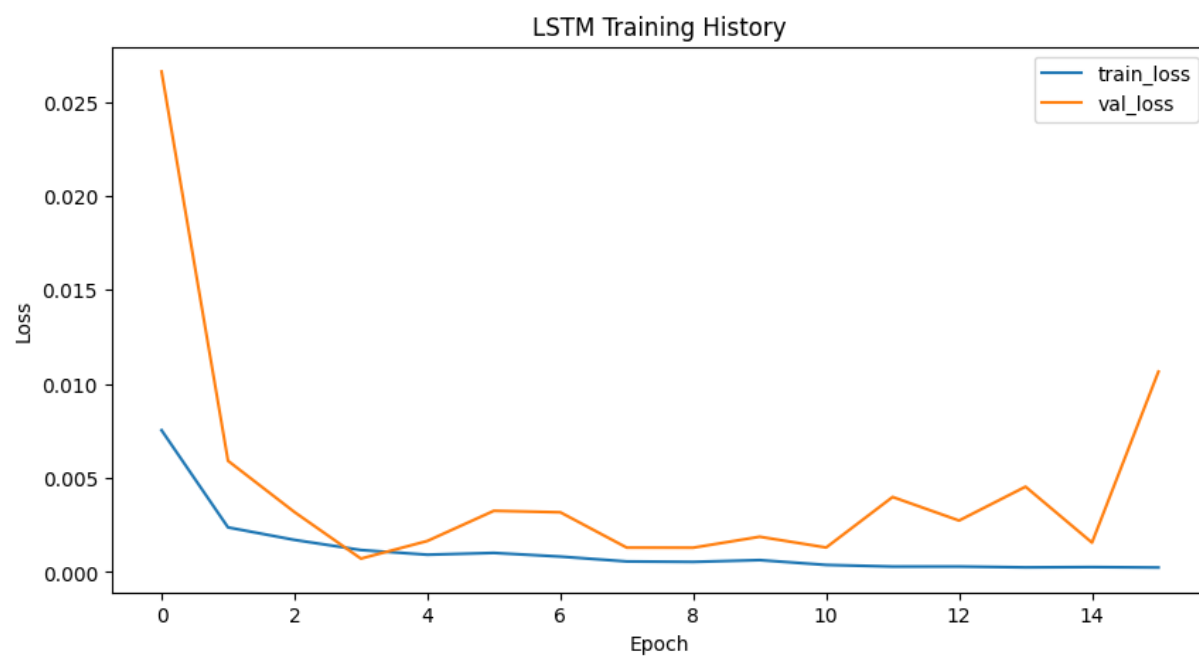
def detect_overfitting(hist):
    train_losses = hist.history['loss']
    val_losses = hist.history.get('val_loss')
    if val_losses is None:
        return 'Không có validation loss để đánh giá.'
    min_val_epoch = np.argmin(val_losses)
    min_val = val_losses[min_val_epoch]
    final_train = train_losses[-1]
    final_val = val_losses[-1]
    messages = []
    # Dấu hiệu: val_loss tăng trong khi train_loss giảm
    if final_train < train_losses[0] and final_val > min_val:
        messages.append('Có dấu hiệu overfitting: train loss tiếp tục giảm trong khi validation loss đã tăng.')
    # Dấu hiệu: chênh lệch lớn giữa train và val
    if (final_val - final_train) / (abs(final_train) + 1e-8) > 0.2:
        messages.append('Chênh lệch lớn giữa validation loss và train loss - Khả năng overfitting cao.')
    # Kiểm tra epoch tốt nhất
    messages.append(f'Epoch có val_loss nhỏ nhất: {min_val_epoch}, val_loss tại đó = {min_val:.6f}.')
    return '\n'.join(messages)

print('RNN overfit check:\n', detect_overfitting(hist_rnn))
print('LSTM overfit check:\n', detect_overfitting(hist_lstm))

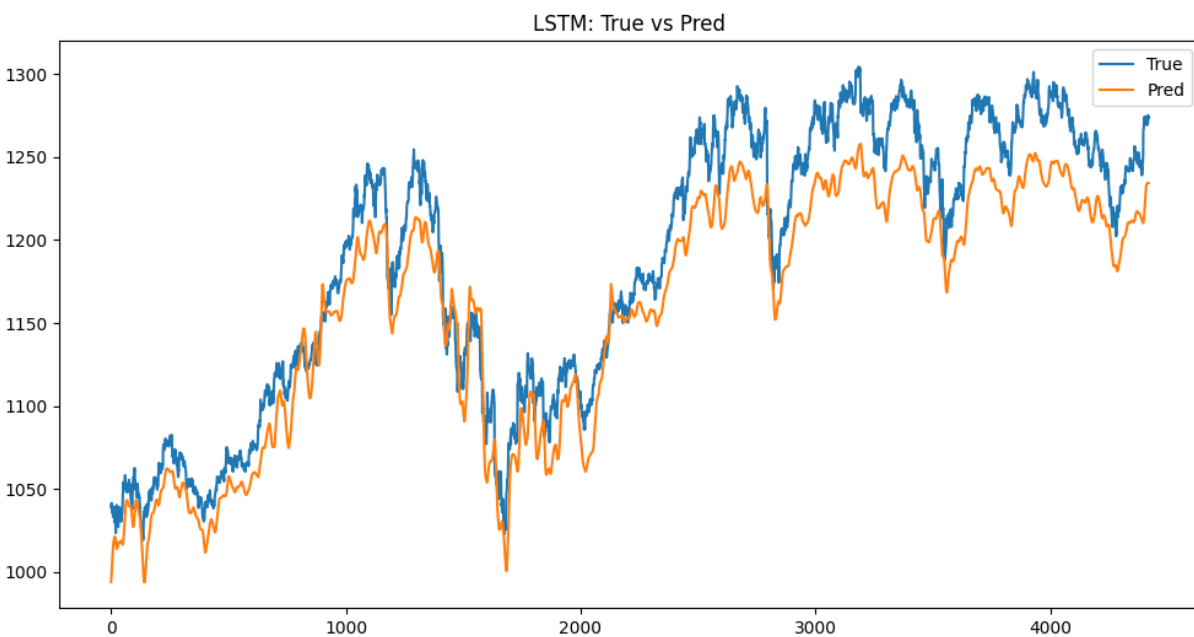
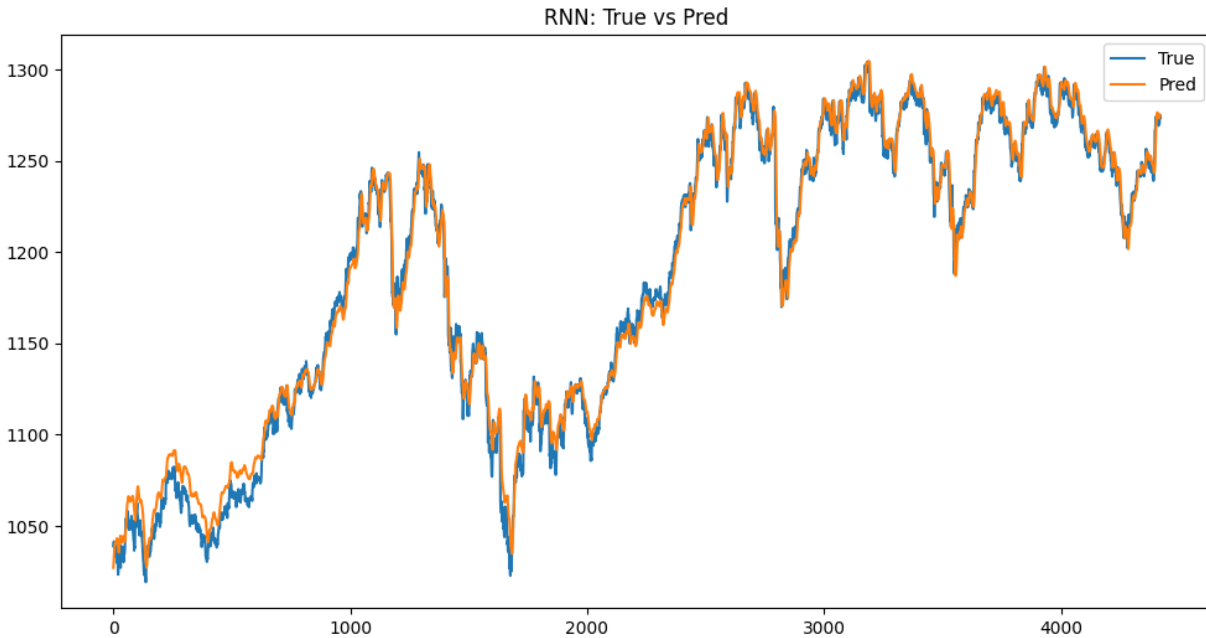
```



139/139 ————— 3s 18ms/step
RMSE: 8.2541, MAE: 6.4108



139/139 ————— 6s 39ms/step
RMSE: 30.1169, MAE: 27.4014



RNN overfit check:

Có dấu hiệu overfitting: train loss tiếp tục giảm trong khi validation loss đã tăng.
Chênh lệch lớn giữa validation loss và train loss – khả năng overfitting cao.
Epoch có val_loss nhỏ nhất: 13, val_loss tại đó = 0.000170.

LSTM overfit check:

Có dấu hiệu overfitting: train loss tiếp tục giảm trong khi validation loss đã tăng.
Chênh lệch lớn giữa validation loss và train loss – khả năng overfitting cao.
Epoch có val_loss nhỏ nhất: 3, val_loss tại đó = 0.000716.

- Cách phát hiện overfitting:

- Quan sát đồ thị train_loss và val_loss theo epoch:
 - Nếu train_loss liên tục giảm nhưng val_loss bắt đầu tăng lại → mô hình đang học quá tốt trên dữ liệu huấn luyện (memorize), mất khả năng khái quát hóa → overfitting.
- So sánh hiệu năng trên tập test (RMSE/MAE): nếu chênh lệch lớn giữa train vs test → dấu hiệu overfitting.
- Ngoài ra, dự báo multi-step (iterative) thường phóng đại sai số; nếu error nhanh tăng khi forecast horizon tăng → mô hình quá nhạy/không ổn định.

- Hiện tượng overfitting quan sát được:

- Loss train giảm liên tục
- Loss validation đi ngang hoặc tăng sau một số epoch
- Đồ thị dự báo bám quá sát train nhưng lệch mạnh test
- Mô hình RNN overfit nhiều hơn LSTM

- Nguyên nhân overfit:

1. Mô hình quá sâu (≥ 7 layer) → quá nhiều tham số.
2. Dữ liệu chứng khoán nhiễu → mô hình học nhiễu.
3. Không sử dụng dropout hoặc regularization.
4. Không chuẩn hóa tính năng phù hợp.
5. Dừng cửa sổ thời gian chưa tối ưu.

- Giải pháp chống overfit:

- Regularization: Dropout giữa các layers (đã dùng), L2 weight decay (thêm nếu cần).
- EarlyStopping: dừng khi val_loss không cải thiện (đã dùng).
- Giảm kích thước mô hình: giảm số units, giảm số layers, thử 3–4 layers thay vì 7 nếu dữ liệu nhỏ.

- Data augmentation / smoothing: resample, rolling-averages, sử dụng returns thay vì giá tuyệt đối.
- Feature engineering: thêm biến macro, volume, indicators (MA, RSI) để tăng thông tin hữu ích.
- Cross-validation theo time (time-series CV) để chọn hyperparams.
- Ensembling: trung bình dự báo của nhiều mô hình để giảm varian.
- Multi-step direct vs iterative: thay vì iterative forecast dài hạn, huấn luyện mô hình cho trực tiếp k-step ahead (multi-output) để giảm tích tụ sai số.
- Standardize problem: dự báo returns/log-returns thường ổn định hơn dự báo giá tuyệt đối.
- Dùng mô hình regularized LSTM như:
 - Bidirectional LSTM
 - GRU

- Chọn mô hình tốt nhất để đưa ra dự báo theo ngày, tháng, năm

```
[11]: # %%
# Dự BÁO (Ví dụ): 1-step ahead (ngày tiếp theo) dùng model tốt nhất
# Chọn model tốt nhất bằng RMSE đơn giản
if rmse_rnn < rmse_lstm:
    best_model = rnn_model
    print('Chọn RNN là mô hình tốt nhất (dựa trên RMSE).')
else:
    best_model = lstm_model
    print('Chọn LSTM là mô hình tốt nhất (dựa trên RMSE).')

# Lấy sequence cuối cùng từ toàn bộ dữ liệu để dự đoán bước tiếp theo
last_sequence = scaled[-SEQUENCE_LENGTH:]
last_seq_resaped = last_sequence.reshape((1, SEQUENCE_LENGTH, 1))
next_pred_scaled = best_model.predict(last_seq_resaped)
next_pred = scaler.inverse_transform(next_pred_scaled).flatten()[0]
print(f'Forecast 1-step ahead (next day): {next_pred:.4f}')

Chọn RNN là mô hình tốt nhất (dựa trên RMSE).
1/1 ————— 0s 81ms/step
Forecast 1-step ahead (next day): 1275.6394
```



```
[13]: def multi_step_forecast(model, last_sequence_scaled, k=30, scaler=scaler):
# nếu là numpy array, chuyển sang list
if isinstance(last_sequence_scaled, np.ndarray):
    seq = last_sequence_scaled.copy().tolist()
else:
    seq = last_sequence_scaled.copy()
preds = []
for _ in range(k):
    arr = np.array(seq[-SEQUENCE_LENGTH:]).reshape((1, SEQUENCE_LENGTH, 1))
    p = model.predict(arr, verbose=0)[0,0]
    preds.append(p)
    seq.append([p])
preds = np.array(preds).reshape(-1,1)
return scaler.inverse_transform(preds).flatten()
```

Dự BÁO 30 NGÀY

```
pred_30_days = multi_step_forecast(best_model, list(scaled), k=30)
print('Dự báo 30 ngày đầu tiên:', pred_30_days[:10])
```

Dự BÁO 1 NĂM (365 NGÀY)

```
pred_1_year = multi_step_forecast(best_model, list(scaled), k=365)
print('Dự báo 1 năm đầu tiên:', pred_1_year[:10])
```

```
Dự báo 30 ngày đầu tiên: [1275.6394 1276.306 1277.0219 1277.7766 1278.6342 1279.4645 1280.302
1281.1531 1281.9922 1282.8405]
Dự báo 1 năm đầu tiên: [1275.6394 1276.306 1277.0219 1277.7766 1278.6342 1279.4645 1280.302
1281.1531 1281.9922 1282.8405]
```